چکیده

در این پروژه، طراحی و تحلیل سیستم کنترلی کوادروتور به‌عنوان یک سیستم دینامیکی غیرخطی با بهره‌گیری از روش‌های پیشرفته کنترل انجام شده است. در فاز اول، مدل غیرخطی سیستم استخراج و با خطی‌سازی حول نقطه تعادل، مدل فضای حالت به دست آمد. تحلیل‌هایی شامل پایداری، کنترل‌پذیری و رویت‌پذیری سیستم انجام شد و محدوده اعتبار مدل خطی با شبیه‌سازی بررسی گردید. در فاز دوم، کنترل‌کننده‌های فیدبک حالت و LQR با هدف بهبود پایداری و ردگیری ورودی مرجع طراحی شدند. همچنین، پیش‌جبران‌سازهای استاتیکی و دینامیکی برای کاهش اثر اغتشاش و بهبود عملکرد سیستم افزوده شدند. در فاز سوم، رویت‌گرهای مرتبه کامل و کاهش‌یافته برای تخمین حالت‌ها در شرایط عدم دسترسی به متغیرهای کامل طراحی شدند. کنترل سیستم با استفاده از حالت‌های تخمین زده‌شده در مواجهه با نویز اندازه‌گیری، اشباع عملگر و عدم قطعیت پارامتری بررسی شد. علاوه بر این، فیلتر کالمن برای بهبود تخمین حالت‌ها و گسسته‌سازی سیستم کنترلی جهت پیاده‌سازی عملی مورد استفاده قرار گرفت. نتایج شبیه‌سازی نشان داد که طراحی‌های انجام‌شده، عملکرد پایدار و دقیق سیستم کوادروتور را در شرایط عملیاتی مختلف تضمین کرده و به دستیابی به خروجی‌های مطلوب کمک می‌کنند.

واژه‌های کلیدی: کوادروتور، مدل غیرخطی، خطی‌سازی، پایداری لیاپانوف، کنترل‌پذیری، رویت‌پذیری، تجزیه کالمن، فیدبک حالت، کنترلر LQR، پیش‌جبران‌ساز، رویت‌گر، شبیه‌سازی، سیمولینک

فهرست مطالب

صفحه

فصل اول: مقدمه ۱
۱-۱- مقدمه ۲
۱-۲- معرفی سیستم ۲
۱-۲-۱- بدنه و ساختار (Frame and Structure) ۳
۱-۲-۲- موتورها و پروانه‌ها (Motors and Propellers) ۳
۱-۲-۳- حسگرها (Sensors) ۴
۱-۲-۴- سیستم‌های کنترلی و عملگرها (Control Systems and Actuators) ۵
۱-۲-۵- سیستم کنترل پرواز (Flight Controller) ۵
۱-۳- مدل غیرخطی ۶
۱-۳-۱- پارامترهای سیستم ۷
فصل دوم: توضیح کد متلب و شبیه‌سازی سیمولینک ۸
۲-۱- تعریف متغیرهای سیمبولیک: ۹
۲-۲- مدل‌سازی حرکت ترجمه‌ای و دورانی کوادروتور ۱۱
۲-۳- خطی‌سازی سیستم کوادروتور ۱۳
۲-۴- محاسبه تابع تبدیل سیستم خطی‌شده: ۱۴
۲-۵- پایداری لیاپانوف: ۱۵
۲-۶- تحلیل Kalman Decomposition ۱۸
۲-۷- ساخت و کاهش مدل فضای حالت ۱۸
۲-۸- طراحی کنترل‌کننده LQR: ۲۲
۲-۹- طراحی پیش‌جبران‌کننده استاتیکی ۲۳
۲-۱۰ طراحی پیش‌جبران‌کننده دینامیکی با و بدون کنترل‌کننده LQR ۲۴
۲-۱۱- بلوک control input generator ۲۵
۲-۱۲ بلوک actuator signal: ۲۷
۲-۱۳ طراحی مشاهده‌گر مرتبه کامل : ۲۸
۲-۱۴-طراحی مشاهدگر مرتبه کاهش یافته: ۲۹
فصل سوم: پیاده‌سازی فاز اول و دوم ۳۲
۳-۱- پیاده‌سازی در سیمولینک (پاسخ سوالات ۸ و ۹ فاز اول) ۳۳
3-2- ماتریس انتقال حالت ۳۸
۳-۳- تعیین شرایط اولیه (حذف فرکانس) ۴۲
۳-۴- بررسی پایداری BIBO: ۴۲

۳-۵- تحلیل حساسیت سیستم نسبت به اغتشاشات ۴۲

فصل چهارم: فاز سوم ۴۳

۴-۱-Dynamic Pre-compensator with LQR ۴۴

۴-۲- Actuator Signal ۴۶

۴-۳- رویتگر ها ۴۸

۴-۴- وجود نویز ۵۴

۴-۵- حضور عدم قطعیت های پارامتری ۵۶

فصل پنجم: جمع‌بندی، نتیجه‌گیری و پیشنهادات ۵۸

منابع و مراجع ۶۰

فهرست اشکال

صفحه

شکل ۱-۱ نمونه‌ای از یک کوادروتور ۲
شکل ۲-۱ سیستم کوادروتور ۶ درجه آزادی..... ۶
شکل ۸-۴ خروجی سیمولینک برای شرایط اولیه یک و ورودی صفر ۳۳
شکل ۹-۴ خروجی سیمولینک برای شرایط اولیه صفر و ورودی ۰.۲ ..... ۳۴
شکل ۱۰-۴ خروجی سیمولینک برای delta u1 = 5..... ۳۴
شکل ۱۱-۴ خروجی سیمولینک برای delta u2 = 0.05..... ۳۵
شکل ۱۲-۴ خروجی سیمولینک برای delta u3 = 0.05..... ۳۵
شکل ۱۳-۴ خروجی سیمولینک برای delta u4 = 0.05..... ۳۶
شکل ۱۴-۴ حالت خطی با انتگرال‌گیر ۳۶
شکل ۱۵-۴ حالت خطی با انتگرال‌گیر، شرایط اولیه صفر و ورودی پله ۰.۲..... ۳۷
شکل ۱۸-۴ ماتریس انتقال حالت در حوزه لاپلاس..... ۳۹
شکل ۱۹-۴ محاسبه ماتریس انتقال حالت در حوزه زمان ۴۰
شکل ۲۰-۴ محاسبه خروجی سیستم در حوزه زمان..... ۴۰
شکل ۲۱-۴ پاسخ خروجی سیستم به شرط اولیه دلخواه و ورودی پله واحد..... ۴۱

فهرست جداول

صفحه

جدول ۲-۱- پارامترهای مورد نیاز برای مدلسازی کوادروتور ۷

فصل اول:

مقدمه

۱-۱- مقدمه

در این پروژه، به تحلیل سیستم کوادروتور با رویکردهایی که در کنترل آموختیم، می‌پردازیم. این گزارش شامل توضیح کد متلب، شرح کامل شبیه‌سازی سیمولینک، و پاسخ‌گویی به سوالات مطرح شده در فاز اول تا سوم می‌باشد. در ادامه، سیستم موردنظر را معرفی کرده، اجزای آن را بررسی و نحوه مدلسازی آن را تحلیل می‌کنیم.

۱-۲- معرفی سیستم

کوادروتور به عنوان یکی از سیستم‌های پیچیده کنترل در پرواز، ساختاری با اجزای متنوع دارد که هر کدام در مدیریت و حفظ تعادل دستگاه نقش دارند. این سیستم به دلیل نیازهای کنترلی خاص خود، یکی از نمونه‌های مهم برای پیاده‌سازی و ارزیابی روش‌های کنترل مدرن مانند PID و LQR است. در ادامه، به شرح جزئیات اجزای فیزیکی و نحوه عملکرد کوادروتور می‌پردازیم.



شکل ۱-۱ نمونه‌ای از یک کوادروتور

۱-۲-۱- بدنه و ساختار (Frame and Structure)

بدنه کوادروتور به گونه‌ای طراحی شده است که در عین سبکی، استحکام بالایی داشته باشد و بتواند تنش‌های وارده از چرخش موتورها و پروانه‌ها را تحمل کند. در ساخت بدنه کوادروتور، معمولاً از مواد مقاوم و سبک مانند فیبر کربن یا آلومینیوم در ساخت بدنه استفاده می‌شود. این بدنه شامل چهار بازو است که به صورت متقارن به مرکز بدنه متصل شده‌اند و پروانه‌ها و موتورها در انتهای هر بازو قرار دارند. این طراحی متقارن امکان حفظ تعادل دستگاه را فراهم کرده و همچنین توزیع بار بر روی کل ساختار بهینه می‌شود.

۱-۲-۲- موتورها و پروانه‌ها (Motors and Propellers)

موتورها و پروانه‌های کوادروتور به عنوان عملگرهای اصلی، نیروهای پیشرانش و گشتاورهای لازم برای حرکت را فراهم می‌کنند. در کوادروتور، چهار موتور به گونه‌ای قرار گرفته‌اند که دو موتور در جهت ساعتگرد و دو موتور دیگر در جهت پادساعتگرد بچرخند. این ترکیب چرخش موجب خنثی شدن گشتاورهای ناخواسته و جلوگیری از چرخش غیرکنترلی دستگاه می‌شود. تنظیم سرعت هر موتور باعث تغییر وضعیت و حرکت کوادروتور در جهات مختلف می‌شود. برای مثال، افزایش سرعت چرخش پروانه‌های عقب و کاهش سرعت پروانه‌های جلو باعث حرکت به سمت جلو می‌شود.

۱-۲-۳- حسگرها (Sensors)

سیستم کنترل پرواز کوادروتور از مجموعه‌ای از حسگرها برای جمع‌آوری اطلاعات مربوط به وضعیت و موقعیت دستگاه استفاده می‌کند. حسگرهای اصلی که در کوادروتور استفاده می‌شوند، عبارتند از:

- شتاب‌سنج (Accelerometer): این حسگر میزان شتاب خطی دستگاه را در جهات مختلف اندازه‌گیری می‌کند. شتاب‌سنج کمک می‌کند که سیستم کنترل پرواز بتواند تغییرات سرعت را تشخیص دهد و از جابه‌جایی ناگهانی جلوگیری کند.
- ژیروسکوپ (Gyroscope): ژیروسکوپ برای اندازه‌گیری سرعت زاویه‌ای و حفظ تعادل زاویه‌ای دستگاه در پرواز استفاده می‌شود. این حسگر اطلاعات دقیقی در مورد چرخش‌ها و انحرافات ارائه می‌دهد که برای کنترل پایداری ضروری است.
- مغناطیس‌سنج (Magnetometer): این حسگر جهت‌گیری دستگاه نسبت به میدان مغناطیسی زمین را اندازه‌گیری می‌کند و کمک می‌کند که کوادروتور در مسیر تعیین‌شده به صورت دقیق حرکت کند.
- حسگر فشار و ارتفاع (Altimeter/Barometer): این حسگر ارتفاع کوادروتور از سطح زمین را اندازه‌گیری می‌کند و به کنترلر امکان می‌دهد که پرواز را در ارتفاع معین و ثابتی نگه دارد. این ویژگی برای برخاست و فرود خودکار ضروری است.
- حسگر GPS: این حسگر برای تعیین موقعیت جغرافیایی دقیق کوادروتور به‌کار می‌رود.

۱-۲-۴- سیستم‌های کنترلی و عملگرها (Control Systems and Actuators)

عملگرهای کوادروتور شامل موتورها و سیستم‌های الکترونیکی کنترل آن‌ها می‌باشند. این عملگرها دستورات را از سیستم کنترل پرواز دریافت کرده و با تغییر سرعت چرخش پروانه‌ها، نیروی لازم برای حرکت در جهت‌های مختلف را ایجاد می‌کنند. عملگرها به کمک حسگرها، پاسخ‌های کنترلی را در زمان واقعی به وضعیت دستگاه می‌دهند و به حفظ تعادل و مانورپذیری کمک می‌کنند.

۱-۲-۵- سیستم کنترل پرواز (Flight Controller)

سیستم کنترل پرواز، هسته مرکزی مدیریت حرکت و پایداری کوادروتور است. این سیستم وظیفه دارد داده‌های حسگرها را پردازش کرده و دستورات حرکتی را برای موتورها ارسال کند. دو روش کنترلی متداول برای کوادروتور شامل کنترل PID و کنترل LQR است.

- **کنترل PID**:

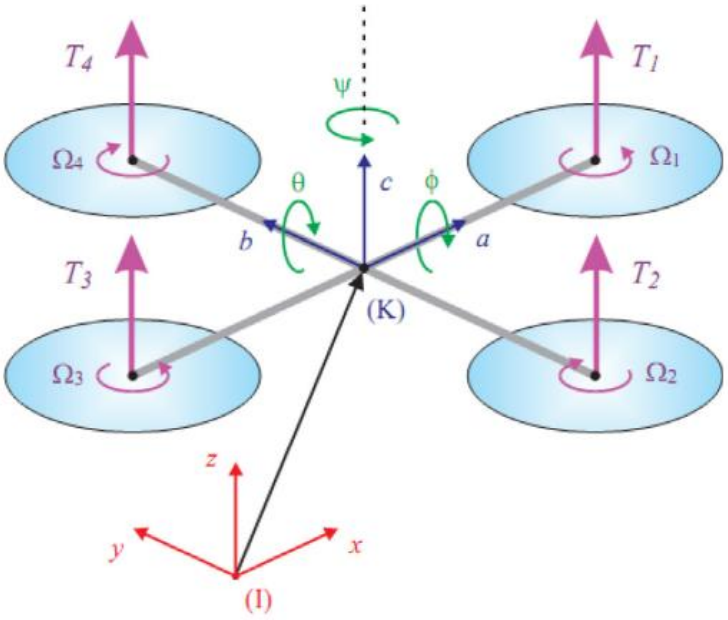
کنترلر PID یک کنترلر کلاسیک است که از سه پارامتر تناسبی (P)، انتگرالی (I)، و مشتقی (D) برای تنظیم خروجی استفاده می‌کند. این کنترلر معمولاً در کوادروتورهای ساده به کار می‌رود، زیرا تنظیم پارامترهای آن ساده‌تر است و می‌تواند پاسخ‌های سریع ارائه دهد. با این حال، این کنترلر در برابر تغییرات ناگهانی و اختلالات بیرونی مقاومت کمتری دارد.

- **کنترل LQR (Linear Quadratic Regulator)**:

کنترلر LQR یک روش بهینه برای کنترل دستگاه است که با استفاده از مدل‌های فضای حالت و تابع هزینه طراحی شده است. این روش توانایی بالایی در مدیریت سیستم‌های غیرخطی و کنترل دقیق‌تر دستگاه در شرایط پیچیده دارد. در کنترل LQR، ماتریس‌های وزنی Q و R به گونه‌ای تنظیم می‌شوند که معیارهای بهینه‌سازی مانند مصرف انرژی و دقت حرکتی بهبود یابند. این کنترلر در شرایط دشوار و محیط‌های پر از اختلال، پایداری بالاتری نسبت به کنترلر PID ارائه می‌دهد.

۱-۳- مدل غیرخطی

مدل غیرخطی از مقاله انتخابی استخراج شده‌است. در ادامه به شرح این پارامترها و خطی‌سازی مدل مذکور می‌پردازیم.



شکل ۲-۱ سیستم کوادروتور ۶ درجه آزادی

۱-۳-۱- پارامترهای سیستم

پارامترهای مورد نیاز جهت مدلسازی کوادروتور به شرح زیر می‌باشند.

جدول ۱-۱- پارامترهای مورد نیاز برای مدلسازی کوادروتور

پارامتر	نمایش (نماد)	مقدار	واحد
جرم کوادروتور	*m (mass)*	*1*	*kg*
شتاب گرانش	*g (gravity)*	*9.81*	*m/s²*
ممان اینرسی حول محور x	*Ixx (moment of inertia x-axis)*	*8.5532E-3*	*kg·m²*
ممان اینرسی حول محور y	*Iyy (moment of inertia y-axis)*	*8.5532E-3*	*kg·m²*
ممان اینرسی حول محور z	*Izz (moment of inertia z-axis)*	*1.476E-2*	*kg·m²*
ثابت ژیروسکوپی	*J (gyroscopic constant)*	*0.00001*	*kg·m²*
ثابت نیروی پیش‌ران‌ها	*b (thrust constant)*	*7.66E-5*	*N·m·s²*
ضریب گشتاور	*d (drag constant)*	*5.63E-6*	*N·m·s²*
فاصله پروانه‌ها از مرکز	*l (distance to center)*	*0.22*	*m*
سرعت موتور در حالت تعادل	*Omega_h (hover motor speed)*	*180*	*rad/s*

فصل دوم:

توضیح کد متلب و شبیه‌سازی سیمولینک

۲-۱- تعریف متغیرهای سیمولیک:

برای شروع ابتدا لازم است که مقادیر عددی ثابت و متغیرهای مربوط به موقعیت، سرعت، زوایا و نرخ‌های زاویه‌ای تعریف شوند. این متغیرها وضعیت لحظه‌ای کوادروتور را توصیف می‌کنند.

Define symbolic variables

```
syms phi theta psi phi_dot theta_dot psi_dot x_dot y_dot z_dot x y z real;
syms p q r p_dot q_dot r_dot real;
syms U1 U2 U3 U4 real;

% Define numerical values for parameters
m = 1;  % mass in kg (example value)
g = 9.81; % gravity in m/s^2
Ixx = 8.5532e-3;  % moment of inertia around x-axis (example value)
Iyy = 8.5532e-3;  % moment of inertia around y-axis (example value)
Izz = 1.476e-2;  % moment of inertia around z-axis (example value)
b = 7.66e-5;
d = 5.63e-6;
l = 0.22;
Omega = 0;
```

x,y,z : موقعیت در دستگاه مختصات.

$\dot{x}, \dot{y}, \dot{z}$: سرعت‌های خطی.

ϕ,θ,ψ : زوایای اویلر (roll, pitch, yaw).

p,q,r : نرخ‌های زاویه‌ای در محورهای مختلف.

سپس لازم است ورودی‌های کنترلی را تعریف کرده و روابط میان آنها را مشخص کنیم:

```
% Define control inputs directly as U1, U2, U3, U4
% U1 = Total thrust
% U2 = Roll torque
% U3 = Pitch torque
% U4 = Yaw torque

% Define relationships between control inputs and motor speeds
% U1_eq = b * (Omega1^2 + Omega2^2 + Omega3^2 + Omega4^2);
% U2_eq = b * (-Omega2^2 + Omega4^2);
% U3_eq = b * (Omega1^2 + Omega3^2);
% U4_eq = d * (-Omega1^2 + Omega2^2 - Omega3^2 + Omega4^2);
```

حالا بردار حالت X که ترکیبی از تمامی متغیرهای حالت است که دینامیک کلی سیستم را توصیف می‌کند را تعریف می‌کنیم.

$$\begin{pmatrix} x \\ x_{dot} \\ y \\ y_{dot} \\ z \\ z_{dot} \\ \varphi \\ \theta \\ \psi \\ p \\ q \\ r \end{pmatrix}$$

```
% Define state vector X (including positions x, y, z and velocities xdot, ydot, zdot)
X = [x , x_dot , y , y_dot , z , z_dot , phi ,theta , psi , p , q ,r];
disp("State vector:")
```

سپس بردار ورودی را تعریف می‌کنیم:

```
U =[U1 , U2 , U3 , U4];
disp("Input vector")
```

```
Input vector
```

```
disp(transpose(U))
```

$$\begin{pmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \end{pmatrix}$$

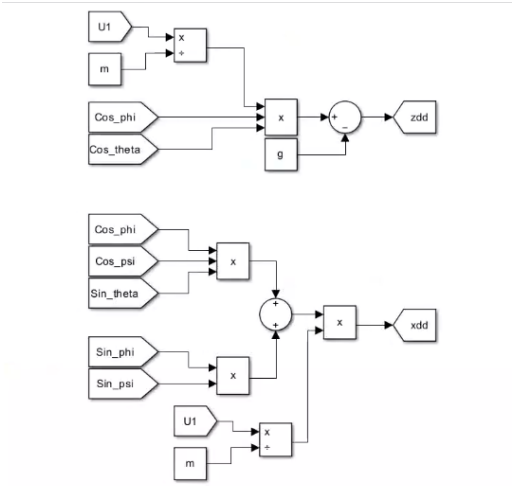
۲-۲- مدل‌سازی حرکت ترنزیشنال و دورانی کوادروتور

ابتدا در بخش معادلات، معادلات مربوطه را پیاده‌سازی می‌کنیم.

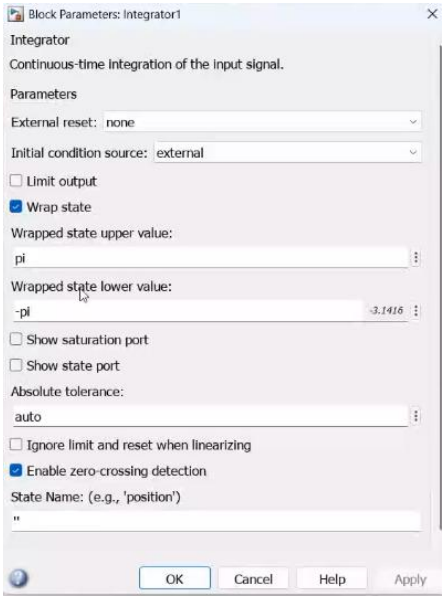
```
% For translational motion
zdd = -g + (cos(phi)*cos(theta)*(1/m)*U1);  % Zdotdot = zdd
xdd = (cos(phi)*sin(theta)*cos(psi) + sin(phi)*sin(psi))*(1/m)*U1;  % xdotdot = xdd
ydd = (cos(phi)*sin(theta)*sin(psi) - sin(phi)*cos(psi))*(1/m)*U1;  % ydotdot = ydd

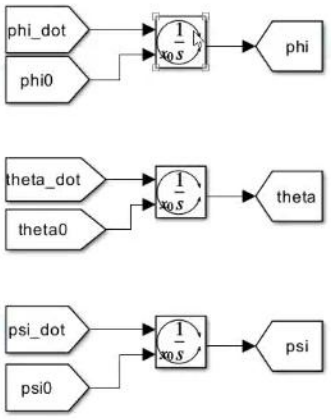
% Define equations of motion (non-linear) using U1, U2, U3, U4
p_dot = q*r*(Iyy - Izz)/Ixx + (1/Ixx)*U2;  % phidotdot = phidd
q_dot = p*r*(Izz - Ixx)/Iyy + (1/Iyy)*U3;    % thetadotdot = thetadd
r_dot = p*q*(Ixx - Iyy)/Izz + (1/Izz)*U4;      % psidotdot = psidd

phi_dot = p+q*sin(phi)*tan(theta)+r*cos(phi)*tan(theta);
theta_dot = q*cos(phi)-r*sin(phi);
psi_dot = q*sin(phi)*sec(theta)+r*cos(phi)*sec(theta);
% State derivatives (use new variables for second derivatives)
f = [x_dot,xdd,y_dot,ydd,z_dot,zdd,phi_dot,theta_dot,psi_dot,p_dot,q_dot,r_dot];
disp("State equations:")
```



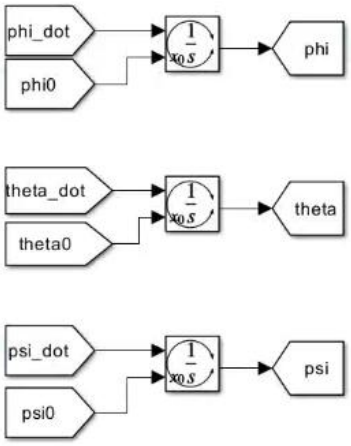
سپس برای ایجاد ارتباط بین مقادیر $x, \dot{x}, \ddot{x}$ و دیگر متغیرها، در بخش state با اضافه کردن دو انتگرال‌گیر، روابط را ایجاد می‌کنیم و مقادیر اولیه را مطابق تصویر زیر، external قرار می‌دهیم.



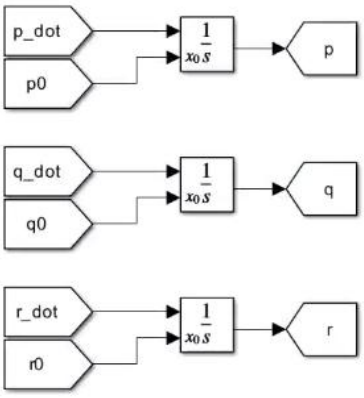


سپس در قسمت بعدی state نیز زاویه‌ها را شبیه‌سازی می‌کنیم و مقدار wrapped state را نیز برابر ۱۸۰- تا ۱۸۰ درجه قرار می‌دهیم.





همچنین برای تعیین p و q و r نیز مطابق زیر عمل می‌کنیم.



۳-۲- خطی‌سازی سیستم کوادروتور

در شرایط هاور، فرضیات زیر برقرار است:

زوایای اویلر (ϕ,θ,ψ) و نرخ‌های زاویه‌ای $(\dot{\psi},\dot{\theta},\dot{\psi})$ برابر با صفر هستند. نیروی رانش کلی (U_1) برابر با وزن سیستم $(m\cdot g)$ است، زیرا این نیرو وزن کوادروتور را در حالت هاور جبران می‌کند. تمامی گشتاورهای کنترلی (U_2,U_3,U_4) صفر هستند و شرایط تعادل به صورت زیر تعریف می‌شود:

```
equilibrium_values = [0, 0, 0, 0, 0, 0, 0, 0, 0 , 0 , 0, m*g, 0, 0, 0];
```

ماتریس A از مشتقات جزئی معادلات دینامیکی f نسبت به x به وجود می‌آید و ماتریس B از مشتقات جزئی معادلات دینامیکی f نسبت به U به وجود می‌آید.

$$B = \frac{\partial f}{\partial U} \qquad A = \frac{\partial f}{\partial x}$$

با جایگذاری شرایط تعادل، A حول نقطه تعادل محاسبه شده و به صورت خطی‌سازی‌شده به دست می‌آید و با جایگذاری شرایط تعادل، B نیز خطی‌سازی‌شده و مقدار آن محاسبه می‌شود.

```
disp(A_eq);
```

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{981}{100} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -\frac{981}{100} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

```
disp(B_eq);
```

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1.0000 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 116.9153 & 0 & 0 \\ 0 & 0 & 116.9153 & 0 \\ 0 & 0 & 0 & 67.7507 \end{pmatrix}$$

همچنین ماتریس C نیز مطابق زیر تعریف شده و ماتریس D را نیز برابر صفر در نظر می‌گیریم.

```
disp(C);
```

```
    1     0     0     0     0     0     0     0     0     0     0     0
    0     0     1     0     0     0     0     0     0     0     0     0
    0     0     0     0     1     0     0     0     0     0     0     0
    0     0     0     0     0     0     0     0     1     0     0     0
```

فرم جردن ماتریس A:

```
Block jordan of matrix A is :
[V,J] = jordan(A_eq);
disp(J)
```

```
    0     1     0     0     0     0     0     0     0     0     0     0
    0     0     1     0     0     0     0     0     0     0     0     0
    0     0     0     1     0     0     0     0     0     0     0     0
    0     0     0     0     0     0     0     0     0     0     0     0
    0     0     0     0     0     1     0     0     0     0     0     0
    0     0     0     0     0     0     0     0     0     0     0     0
    0     0     0     0     0     0     0     1     0     0     0     0
    0     0     0     0     0     0     0     0     1     0     0     0
    0     0     0     0     0     0     0     0     0     0     0     0
    0     0     0     0     0     0     0     0     0     0     0     1
    0     0     0     0     0     0     0     0     0     0     0     0
```

۴-۲- محاسبه تابع تبدیل سیستم خطی‌شده:

معادله کلی تابع تبدیل یک سیستم خطی در حوزه لاپلاس به صورت زیر است:

$$T(s)=C\Phi(s)B$$

$T(s)$: ماتریس تابع تبدیل سیستم (رابطه ورودی-خروجی).

C : ماتریس خروجی که متغیرهای خروجی سیستم را مشخص می‌کند.

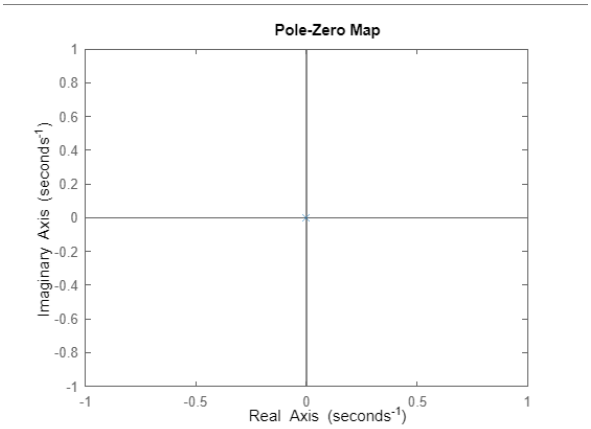
$\Phi(s)$: ماتریس انتقال حالت که رفتار دینامیکی سیستم را در حوزه لاپلاس مدل می‌کند.

B: ماتریس ورودی که تأثیر ورودی‌ها بر سیستم را بیان می‌کند.

```
TF =
```

$$\begin{pmatrix} 0 & 0 & \frac{8070866814559273371}{7036874417766400\,s^4} & 0 \\ 0 & -\frac{8070866814559273371}{7036874417766400\,s^4} & 0 & 0 \\ \frac{1}{s^2} & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{25000}{369\,s^2} \end{pmatrix}$$

نمودار صفر و قطب و مقادیر قطب‌ها:




```
Poles of the system:
disp(poles);
     0
     0
     0
     0
     0
     0
     0
     0
     0
     0
     0
     0
```

همانطور که مشاهده می‌شود تمامی قطب‌ها در صفر قرار گرفته‌اند و سیستم هیچ صفری ندارد.

۲-۵- پایداری لیاپانوف:

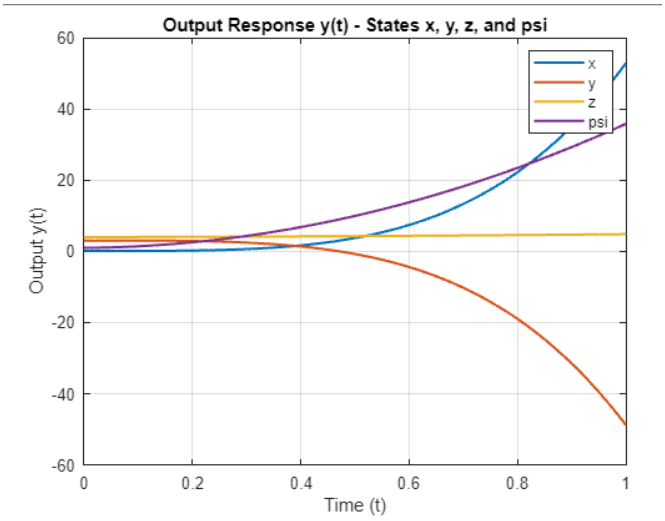
در این بخش پایداری سیستم از نظر لیاپانوف بررسی می‌شود که به دلیل صفر بودن تمامی قطب‌ها امکان پذیر نیست و نمی‌توان به صورت قطعی پاسخ داد.

```
The system is not stable. Lyapunov stability check is not applicable.
```

تحلیل پاسخ خروجی سیستم در حوزه زمان:

این قسمت پاسخ دینامیکی سیستم خطی‌شده را در برابر ورودی‌های پله واحد نشان می‌دهد و تغییرات خروجی‌های کلیدی سیستم (x,y,z,ψ) را در زمان رسم می‌کند. این تحلیل برای بررسی رفتار و پایداری سیستم در شرایط اولیه و ورودی‌های مشخص مفید است.

که همانطور که مشخص است سیستم ناپایدار است و در یک ثانیه به مقدار ۶۰ رسیده است.



محاسبه خروجی سیستم با شرایط اولیه تعریف شده:

این بخش کد خروجی سیستم را با توجه به شرایط اولیه مشخص‌شده محاسبه می‌کند. به‌طور خلاصه، سیستم به‌صورت خطی‌شده مدل‌سازی شده و شرایط اولیه (به‌عنوان ورودی حالت اولیه) برای پیش‌بینی پاسخ سیستم در زمان t به‌کار می‌رود.

```
yout =
```

$$\begin{pmatrix} \frac{327t^3}{1000}+\frac{981t^2}{1000}+\frac{t}{5}+\frac{1}{5} \\ -\frac{327t^3}{1000}-\frac{981t^2}{1000}+\frac{t}{5}+\frac{1}{5} \\ \frac{t}{5}+\frac{1}{5} \\ \frac{t}{5}+\frac{1}{5} \end{pmatrix}$$

بررسی کنترل‌پذیری و رویت پذیری سیستم

این بخش از تحلیل، بررسی کنترل‌پذیری (Controllability) و مشاهده‌پذیری (Observability) سیستم خطی‌شده کوادروتور است. همچنین پایداری و قابلیت تثبیت (Stabilizability) و قابلیت تشخیص (Detectability) سیستم مورد بررسی قرار می‌گیرد.

رنک ماتریس کنترل پذیری و رویت پذیری هر دو برابر ۱۲ می‌باشد؛ پس سیستم هم کنترل‌پذیر و هم رویت‌پذیر می‌باشد.

```
The system is controllable.

The system is observable.
```

همچنین در مرحله بعدی با بررسی Detectability و Stabilizability متوجه می‌شویم که سیستم قابل تشخیص و پیگیری نمی‌باشد.

```
% Check stabilizability
if all(real(eigenvalues(rank(Ctrb) == n)) < 0)
    disp('The system is stabilizable.');
else
    disp('The system is not stabilizable.');
end
```

```
The system is not stabilizable.
```

```
% Check detectability
if all(real(eigenvalues(rank(Obsv) == n)) < 0)
    disp('The system is detectable.');
else
    disp('The system is not detectable.');
end
```

```
The system is not detectable.
```

۲-۶- تحلیل Kalman Decomposition

تحلیل Kalman Decomposition برای جداسازی بخش‌های کنترل‌پذیر و غیرکنترل‌پذیر سیستم خطی‌شده انجام شده است. این فرآیند برای ساده‌سازی تحلیل و طراحی کنترل سیستم استفاده می‌شود.

در این بخش نیز ابتدا رویت‌پذیر و کنترل‌پذیر سیستم مشخص شده و زیر ماتریس‌ها مشخص می‌شوند.

زیرماتریس A_{cc} که دینامیک بخش کنترل‌پذیر سیستم را نمایش می‌دهد و زیرماتریس A_{uu} که دینامیک بخش غیرکنترل‌پذیر سیستم را نمایش می‌دهد.

زیرماتریس B_{cc} که ورودی‌های تأثیرگذار بر بخش کنترل‌پذیر را نشان می‌دهد و زیرماتریس B_{uc} که ورودی‌های تأثیرگذار بر بخش غیرکنترل‌پذیر را نمایش می‌دهد.

زیرماتریس C_{cc} که خروجی‌های مربوط به زیرسیستم کنترل‌پذیر را نشان می‌دهد و زیرماتریس C_{uc} که خروجی‌های مربوط به زیرسیستم غیرکنترل‌پذیر را نمایش می‌دهد.

با توجه به نتایج حاصله، سیستم تنها یک زیر سامانه دارد که کنترل‌پذیر و رویت‌پذیر می‌باشد.

۲-۷- ساخت و کاهش مدل فضای حالت

هدف این بخش، ساخت مدل فضای حالت از سیستم خطی‌شده کوادروتور و کاهش آن به مدل مینیمال (Minimal Realization) است. این فرآیند باعث حذف بخش‌های غیرقابل مشاهده یا غیرکنترل‌پذیر و ساده‌سازی تحلیل و طراحی کنترل می‌شود.

با استفاده از تابع minreal، سیستم به فرم مینیمال کاهش یافته است. هدف این تبدیل، حذف حالت‌های غیرضروری است شامل: زیرسیستم‌های غیرکنترل‌پذیر یا زیرسیستم‌های غیرمشاهده‌پذیر یا کاهش ابعاد ماتریس‌های A، B، C و D.

ماتریس A کاهش یافته:

```
Columns 1 through 9

         0    1.0000         0         0         0         0         0         0         0
         0         0         0         0         0         0         0    9.8100         0
         0         0         0    1.0000         0         0         0         0         0
         0         0         0         0         0         0   -9.8100         0         0
         0         0         0         0         0    1.0000         0         0         0
         0         0         0         0         0         0         0         0         0
         0         0         0         0         0         0         0         0         0
         0         0         0         0         0         0         0         0         0
         0         0         0         0         0         0         0         0         0
         0         0         0         0         0         0         0         0         0
         0         0         0         0         0         0         0         0         0

Columns 10 through 12

         0         0         0
         0         0         0
         0         0         0
         0         0         0
         0         0         0
         0         0         0
    1.0000         0         0
         0    1.0000         0
         0         0    1.0000
         0         0         0
         0         0         0
         0         0         0
```

ماتریس کاهش یافته B:

```
Reduced Input Matrix B:
```

```
disp(B_reduced);
```

```
         0         0         0         0
         0         0         0         0
         0         0         0         0
         0         0         0         0
         0         0         0         0
    1.0000         0         0         0
         0         0         0         0
         0         0         0         0
         0         0         0         0
         0  116.9153         0         0
         0         0  116.9153         0
         0         0         0   67.7507
```

ماتریس کاهش یافته C :

```
Reduced Output Matrix C:
```

```
disp(C_reduced);
```

```
    1     0     0     0     0     0     0     0     0     0     0     0
    0     0     1     0     0     0     0     0     0     0     0     0
    0     0     0     0     1     0     0     0     0     0     0     0
    0     0     0     0     0     0     0     0     1     0     0     0
```

ماتریس کاهش یافته D :

```
Reduced Direct Transmission Matrix D:
disp(D_reduced);
     0     0     0     0
     0     0     0     0
     0     0     0     0
     0     0     0     0
```

ولی خب با این حال احتیاجی به این کار وجود نداشت زیرا سیستم هم رویت‌پذیر و هم کنترل‌پذیر است. پس حتما مینیمال نیز هست.

طراحی فیدبک حالت و شبیه‌سازی پاسخ سیستم

در این بخش می‌خواهیم کنترل‌کننده فیدبک حالت (State Feedback) برای سیستم خطی‌شده کوادروتور به منظور جایگذاری قطب‌های دلخواه و تحلیل پاسخ سیستم در حضور این کنترل‌کننده را طراحی کنیم.

در بخش اول تعیین قطب‌های مطلوب برای سیستم حلقه‌بسته که پایداری و عملکرد مناسب سیستم را تضمین کند مورد نظر می‌باشد قطب‌های با مقدار حقیقی منفی، نشان‌دهنده سیستم پایدار هستند. این قطب‌ها شامل مقادیر حقیقی و مختلط مزدوج‌اند که دینامیک سیستم را مشخص می‌کنند.

از تابع place برای محاسبه ماتریس فیدبک K استفاده شده است.

فرمول حلقه‌بسته:

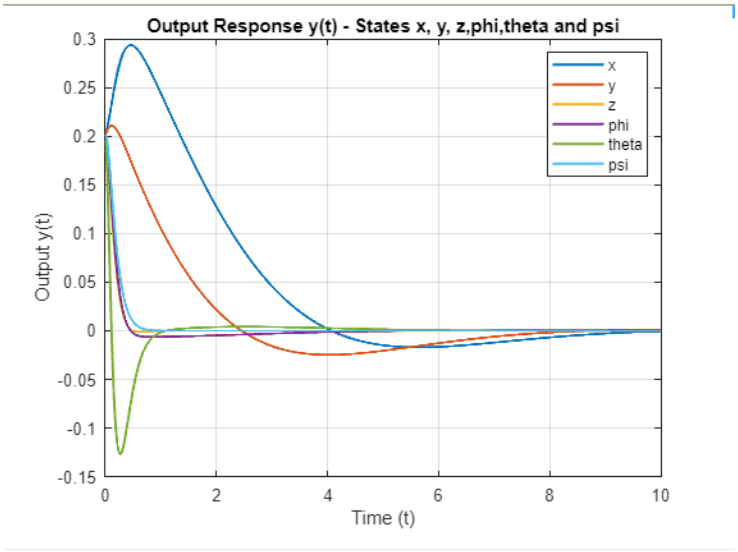
$$A_{feedback} = A_{eq} - B_{eq}k$$

دینامیک سیستم حلقه‌بسته را تعریف می‌کند.

```
Kfeedback =
   11.1746   16.4435  -15.2068   -9.0606   90.1194   18.9907   13.6594   29.9914   -0.0097    0.6270    1.4976   -0.0010
   -0.3570   -0.1202   -0.0064   -0.7780    0.0002    0.0000    2.3772   -0.1312    0.0000    0.2470   -0.0048    0.0000
    0.7452    0.9921   -0.8094   -0.2467    0.0003    0.0000    0.2460    2.5794    0.0000    0.0083    0.2535    0.0000
    0.0179    0.0263   -0.0243   -0.0145   -0.0001   -0.0000    0.0218    0.0479    1.3315    0.0010    0.0024    0.2804
```

```
A_feedback = A_eq - B_eq*Kfeedback
```

```
A_feedback =
         0    1.0000         0         0         0         0         0         0         0         0         0         0
         0         0         0         0         0         0         0    9.8100         0         0         0         0
         0         0         0    1.0000         0         0         0         0         0         0         0         0
         0         0         0         0         0         0   -9.8100         0         0         0         0         0
         0         0         0         0         0    1.0000         0         0         0         0         0         0
  -11.1746  -16.4435   15.2068    9.0606  -90.1194  -18.9907  -13.6594  -29.9914    0.0097   -0.6270   -1.4976    0.0010
         0         0         0         0         0         0         0         0         0    1.0000         0         0
         0         0         0         0         0         0         0         0         0         0    1.0000         0
         0         0         0         0         0         0         0         0         0         0         0    1.0000
   41.7411   14.0503    0.7440   90.9654   -0.0176   -0.0018 -277.9369   15.3397   -0.0019  -28.8771    0.5652   -0.0002
    ⋮
```



طراحی فیدبک حالت به موفقیت انجام شده و سیستم حلقه‌بسته به‌طور مؤثر کنترل می‌شود. خروجی‌ها نشان‌دهنده پایداری سیستم و همگرایی به مقدار مطلوب هستند. نمودار پاسخ سیستم عملکرد کنترل‌کننده را تأیید می‌کند.

۸-۲- طراحی کنترل‌کننده LQR:

ماتریس فیدبک LQR محاسبه شد که به عنوان یک کنترل‌کننده بهینه برای سیستم خطی‌شده عمل می‌کند. مقادیر دقیق آن در خروجی کد نمایش داده می‌شود.

مقادیر ویژه ماتریس $A_{feedback} = A_{eq} - B_{eq}k$ محاسبه شد. تمامی مقادیر ویژه دارای بخش حقیقی منفی هستند، که نشان‌دهنده پایداری سیستم حلقه‌بسته است.

```
K_LQR1 =
    0.0000    0.0000    0.0000    0.0000    5.7735    4.0678   -0.0000    0.0000   -0.0000   -0.0000    0.0000   -0.0000
   -0.0000   -0.0000   -2.2361   -1.9599   -0.0000   -0.0000    6.2328   -0.0000   -0.0000    0.3265   -0.0000   -0.0000
    1.0000    0.9048   -0.0000   -0.0000    0.0000   -0.0000    0.0000    3.0349    0.0000   -0.0000    0.2279    0.0000
   -0.0000   -0.0000   -0.0000   -0.0000    0.0000    0.0000    0.0000   -0.0000    0.8165    0.0000   -0.0000    0.1553
P1_LQR1 = 12×1 complex
  -1.7396
  -1.7396
  -1.7397
  -1.7397
  -2.0339
  -2.0339
  -5.2592
  -5.2592
 -11.5799
 -11.5799
      ⋮
```

```
A_feedback_LQR = A_eq - B_eq*K_LQR1

A_feedback_LQR =
         0    1.0000         0         0         0         0         0         0         0         0         0         0
         0         0         0         0         0         0         0    9.8100         0         0         0         0
         0         0         0    1.0000         0         0         0         0         0         0         0         0
         0         0         0         0         0         0   -9.8100         0         0         0         0         0
         0         0         0         0         0    1.0000         0         0         0         0         0         0
   -0.0000   -0.0000   -0.0000   -0.0000   -5.7735   -4.0678    0.0000   -0.0000    0.0000    0.0000   -0.0000    0.0000
         0         0         0         0         0         0         0         0         0    1.0000         0         0
         0         0         0         0         0         0         0         0         0         0    1.0000         0
         0         0         0         0         0         0         0         0         0         0         0    1.0000
    0.0000    0.0000  261.4306  229.1474    0.0000    0.0000 -728.7095    0.0000    0.0000  -38.1762    0.0000    0.0000
      ⋮
```

ماتریس دینامیک سیستم حلقه‌بسته $A_{feedback}$ به‌صورت عددی محاسبه شد و نشان‌دهنده رفتار بهینه و پایدار سیستم تحت کنترل‌کننده LQR است.

```
eig(A_feedback_LQR)

ans = 12×1 complex
  -11.5799
  -11.5799
  -17.3485
  -17.3485
   -1.7397
   -1.7397
   -5.2592
   -5.2592
   -1.7396
   -1.7396
      ⋮
```

۲-۹- طراحی پیش‌جبران‌کننده استاتیکی

طراحی پیش‌جبران‌کننده استاتیکی (Static Pre-compensator) برای بهبود رفتار خروجی سیستم حلقه‌بسته است. این پیش‌جبران‌کننده با هدف تنظیم ورودی‌های کنترلی برای دستیابی به خروجی مطلوب طراحی شده است.

ماتریس G_0 و شبه‌معکوس آن محاسبه شدند تا ارتباط بین ورودی‌های واقعی و خروجی‌های مطلوب تنظیم شود.

تطبیق ورودی‌ها: پیش‌جبران‌کننده $B_{feedback}$ ورودی‌های سیستم را برای دستیابی به خروجی‌های کلیدی x,y,z,ψ تنظیم می‌کند.

ماتریس انتقال حالت: انتقال حالت حلقه‌بسته $\phi(s)$ برای تحلیل رفتار دینامیکی سیستم محاسبه شد.

۱۰-۲ طراحی پیش‌جبران‌کننده دینامیکی با و بدون کنترل‌کننده LQR

طراحی پیش‌جبران‌کننده استاتیکی (Static Pre-compensator) برای بهبود پاسخ خروجی سیستم حلقه‌بسته استفاده می‌شود و طراحی پیش‌جبران‌کننده دینامیکی (Dynamic Pre-compensator) برای اعمال بهینه‌تر کنترل با استفاده از کنترل‌کننده LQR.

ابتدا G_0 را به عنوان تابع استاتیکی سیستم حلقه بسته سیستم محاسبه می‌کنیم و سپس شبه معکوس G_0 را برای تبدیل به وروی های مورد نیاز محاسبه می‌کنیم.

در بخش با کنترل کننده دینامیک سیستم حلقه‌بسته با ترکیب مدل پیش‌جبران‌کننده و سیستم اصلی را گسترش میدهیم و ماتریس های وزن دهی R و Q را تعریف می کنیم.

```
[K_LQR,~,P_LQR] = lqr(Ai,Bi,Q,R)
```

```
K_LQR =
    0.0000    0.0000   -0.0000    0.0000   33.0337    8.3107   -0.0000    0.0000   -0.0000   -0.0000    0.0000   -0.0000 ···
   -0.0000   -0.0000  -16.5267   -4.5189   -0.0000   -0.0000    7.0284   -0.0000   -0.0000    0.5659   -0.0000    0.0000
   16.6087    5.9446    0.0000    0.0000   -0.0000   -0.0000   -0.0000   11.6150   -0.0000   -0.0000    1.0948    0.0000
    0.0000    0.0000   -0.0000   -0.0000    0.0000    0.0000    0.0000    0.0000    6.6912   -0.0000   -0.0000    1.0943
P_LQR = 16×1 complex
1.0e+02 *

  -0.0182
  -0.0182
  -0.0211
  -0.0211
  -0.0215
  -0.0215
  -0.0320
  -0.0320
  -0.0375
  -0.0375
      ⋮
```

```
K_d_LQR = K_LQR(:,1:12)
```

```
K_d_LQR =
    0.0000    0.0000   -0.0000    0.0000   33.0337    8.3107   -0.0000    0.0000   -0.0000   -0.0000    0.0000   -0.0000
   -0.0000   -0.0000  -16.5267   -4.5189   -0.0000   -0.0000    7.0284   -0.0000   -0.0000    0.5659   -0.0000    0.0000
   16.6087    5.9446    0.0000    0.0000   -0.0000   -0.0000   -0.0000   11.6150   -0.0000   -0.0000    1.0948    0.0000
    0.0000    0.0000   -0.0000   -0.0000    0.0000    0.0000    0.0000    0.0000    6.6912   -0.0000   -0.0000    1.0943
```

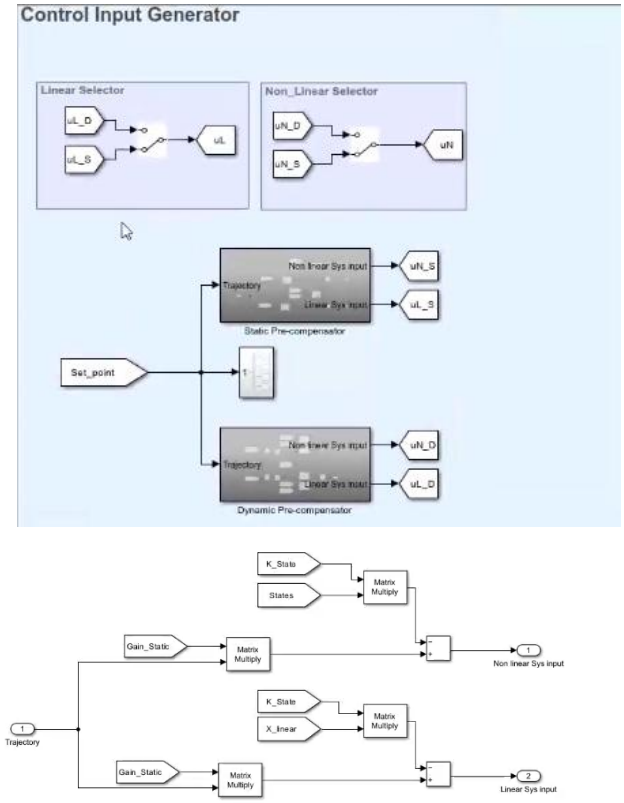
```
K_q_LQR = K_LQR(:,12+1:12+4)
```

```
K_q_LQR =
   -0.0000    0.0000  -63.2456    0.0000
    0.0000   30.0000    0.0000    0.0000
  -22.3607   -0.0000    0.0000    0.0000
   -0.0000    0.0000    0.0000  -20.0000
```

نتیجه می‌گیریم که با کنترل کننده دینامیکی، چون افت رنک داریم، ماتریس های Ai و Bi کنترل‌پذیر و رویت‌پذیر نیستن با وجود آن که سیستم اصلی کنترل‌پذیر و رویت‌پذیر بود.

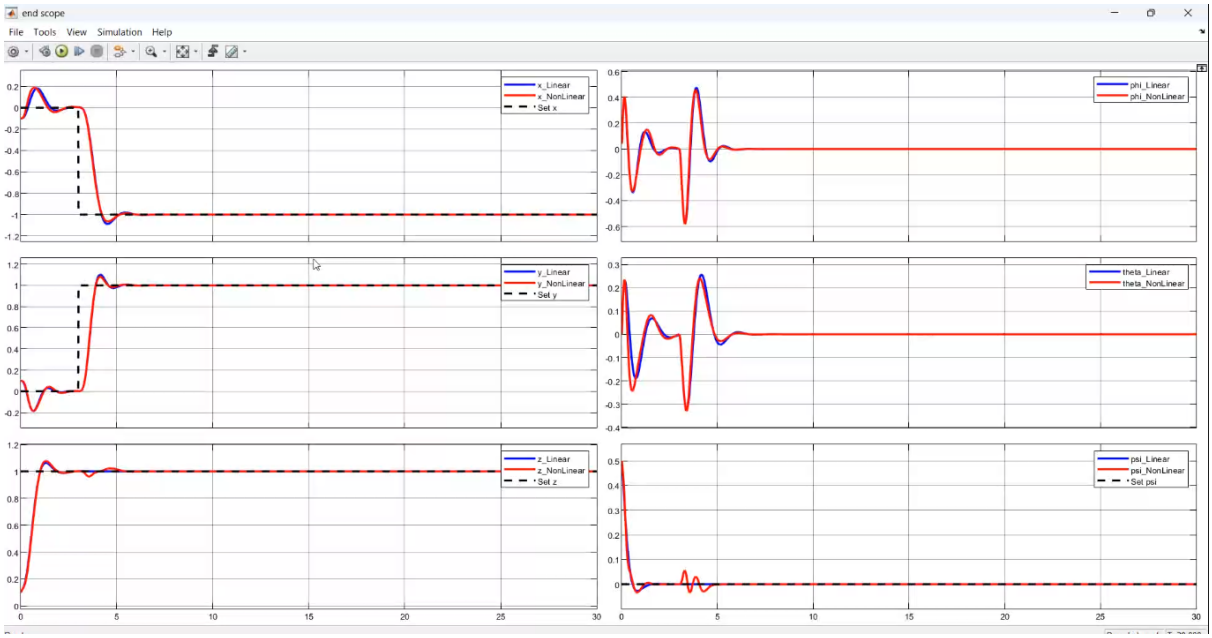
۲-۱۱- بلوک **control input generator** :

در قسمت سیمولینک نیز هر دو کنترل کننده طراحی شده اند که با استفاده از selector ها می‌توانیم نوع خطی یا غیر خطی سیستم و نوع کنترل کننده را انتخاب کنیم.

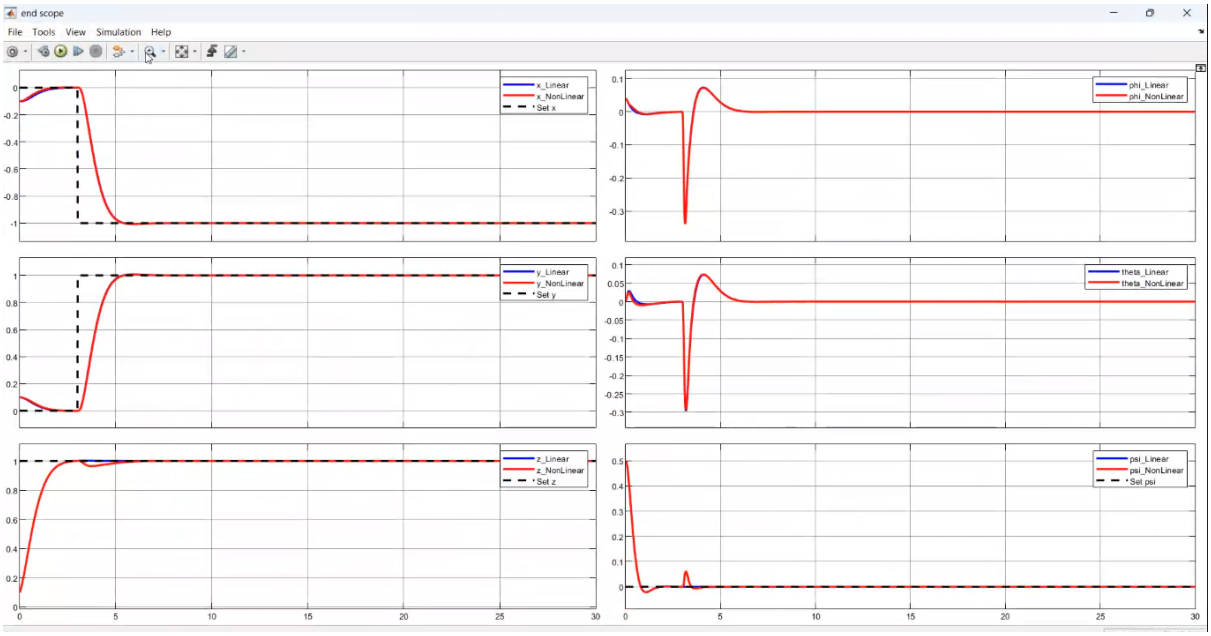


Static Pre-compensator With LQR

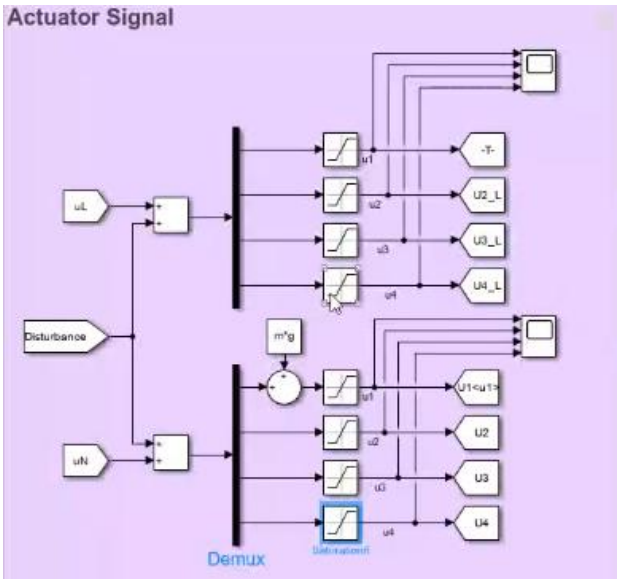
مثلا برای سیستم غیر خطی با کنترل کننده دینامیکی نتایج زیر را داریم که کاملا کنترل شده و مناسب است.



برای کنترل کننده استاتیکی نیز نتایج زیر را داریم:

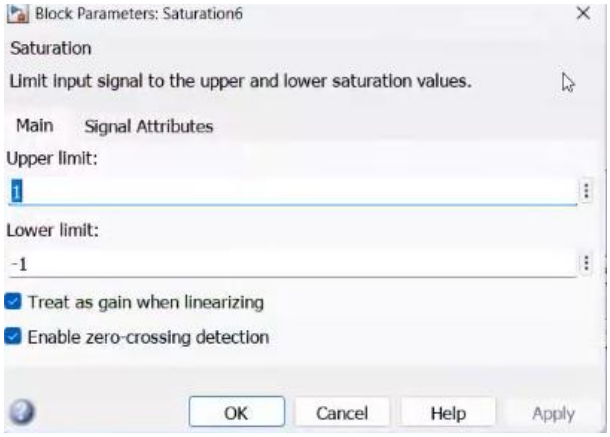


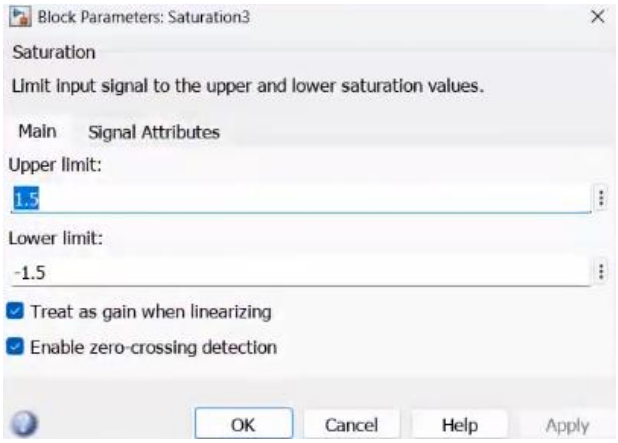
۱۲-۲ بلوک actuator signal:



در این بخش ابتدا حدود مورد نیاز را به صورت دستی محاسبه کرده و سپس در بلوک های saturation مقادیر را قرار می‌دهیم.

دو نمونه برای مثال:





۱۳-۲ طراحی مشاهده‌گر مرتبه کامل :

طراحی و پیاده‌سازی مشاهده‌گر مرتبه کامل برای سیستم خطی‌شده، با هدف تخمین دقیق متغیرهای حالت در شرایط عدم دسترسی به تمامی متغیرها انجام می‌شود.

ابتدا ماتریس های سیستم را تعریف می‌کنیم

ماتریس خروجی C_{eq} با انتخاب خروجی‌های کلیدی x,y,z,ψ و poles_obsv قطب‌های مطلوب برای مشاهده‌گر، با انتخاب مقادیر با بخش حقیقی منفی برای پایداری.

سپس ماتریس L به‌گونه‌ای طراحی میکنیم که قطب‌های دینامیک مشاهده‌گر با مقادیر مطلوب جای‌گذاری شوند.

```
L =
1.0e+04 *

    0.0084    0.0000   -0.0000   -0.0000
    0.2676    0.0000   -0.0000   -0.0000
   -0.0000    0.0084   -0.0000    0.0000
   -0.0000    0.2676   -0.0000    0.0000
    0.0000    0.0000    0.0042   -0.0001
    0.0000    0.0000    0.0450   -0.0017
    0.0000   -0.3839    0.0000   -0.0000
    0.3839    0.0000   -0.0000   -0.0000
   -0.0000   -0.0000   -0.0001    0.0042
    0.0000   -2.0251    0.0000   -0.0000
      ⋮
```

سپس دینامیک مشاهده‌گر را نمایش میدهیم:

```
A_eq=L*C_eq

ans =
1.0e+04 *

   -0.0084    0.0001   -0.0000         0    0.0000         0         0         0    0.0000         0         0         0
   -0.2676         0   -0.0000         0    0.0000         0         0    0.0010    0.0000         0         0         0
    0.0000         0   -0.0084    0.0001    0.0000         0         0         0   -0.0000         0         0         0
    0.0000         0   -0.2676         0    0.0000         0   -0.0010         0   -0.0000         0         0         0
   -0.0000         0   -0.0000         0   -0.0042    0.0001         0         0    0.0001         0         0         0
   -0.0000         0   -0.0000         0   -0.0450         0         0         0    0.0017         0         0         0
   -0.0000         0    0.3839         0   -0.0000         0         0         0    0.0000    0.0001         0         0
   -0.3839         0   -0.0000         0    0.0000         0         0         0    0.0000         0    0.0001         0
    0.0000         0    0.0000         0    0.0001         0         0         0   -0.0042         0         0    0.0001
   -0.0000         0    2.0251         0   -0.0000         0         0         0    0.0000         0         0         0
      ⋮
```

۲-۱۴- طراحی مشاهدگر مرتبه کاهش یافته:

ابتدا رتبه سیستخم خروجی را تعریف می‌کنیم رتبه ماتریس خروجی، تعداد متغیرهای قابل مشاهده مستقیم از سیستم را مشخص می‌کند.

سپس ماتریس دینامیک کاهش یافته F را تعریف می‌کنیم و ماتریس بهره مشاهده‌گر را محاسبه میکنیم.

```
L_R0 =
         300         200         100         500
        -500         200         250         630
        -200       -1000         560         120
        1100        -300         300         500
        -200         250         300         560
        -900         250         700         900
         200         275         500         900
         200        -700         900         250
```

```
C
```

```
C =
     1     0     0     0     0     0     0     0     0     0     0     0
     0     0     1     0     0     0     0     0     0     0     0     0
     0     0     0     0     1     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     1     0     0     0
```

```
T_0 = lyap(-F,A_eq,-L_R0*C)  % T_(n-q)*n
```

```
T_0 =
   58.9354   -6.5527  -15.9218    2.3831  -45.5464    7.9073    1.7182    5.2746  -47.7730   -0.0804   -0.3601    9.4221
   -3.3894    0.2623  -18.8082    1.8976   90.7959  -14.3901    1.2743   -1.2921  122.9780   -0.0970    0.2004  -17.9919
   16.3554   -0.3713  -51.5827    2.1838   88.5296  -11.4753    0.8659   -0.9723   90.4160   -0.0536    0.1496  -13.3805
   29.7544    0.2491   -3.1332   -0.3716   52.2094   -6.3373   -0.2415   -1.0588   74.0701   -0.0049    0.0970   -8.0789
  -45.2420    3.4487   22.5772   -1.1624   69.9992   -7.4047   -0.2390   -1.9865   94.2837   -0.0262    0.0998   -8.8858
  -36.3671    1.3341   10.7845   -0.1260   53.6790   -3.9337    0.2710   -0.3980   65.3661   -0.0354    0.0064   -4.2794
   13.6290   -0.8351   -2.3526    0.8138   42.7258   -2.8928    0.6927    0.4729   47.2688   -0.0472   -0.0263   -2.4501
    9.3023   -0.4327  -32.5581    1.5143   41.8605   -1.9470    0.6910    0.1974   11.6279   -0.0321   -0.0092   -0.5408
```

```
mat2 = [C; T_0]
```

```
mat2 =
    1.0000         0         0         0         0         0         0         0         0         0         0         0
         0         0    1.0000         0         0         0         0         0         0         0         0         0
         0         0         0         0    1.0000         0         0         0         0         0         0         0
         0         0         0         0         0         0         0         0    1.0000         0         0         0
   58.9354   -6.5527  -15.9218    2.3831  -45.5464    7.9073    1.7182    5.2746  -47.7730   -0.0804   -0.3601    9.4221
   -3.3894    0.2623  -18.8082    1.8976   90.7959  -14.3901    1.2743   -1.2921  122.9780   -0.0970    0.2004  -17.9919
   16.3554   -0.3713  -51.5827    2.1838   88.5296  -11.4753    0.8659   -0.9723   90.4160   -0.0536    0.1496  -13.3805
   29.7544    0.2491   -3.1332   -0.3716   52.2094   -6.3373   -0.2415   -1.0588   74.0701   -0.0049    0.0970   -8.0789
  -45.2420    3.4487   22.5772   -1.1624   69.9992   -7.4047   -0.2390   -1.9865   94.2837   -0.0262    0.0998   -8.8858
  -36.3671    1.3341   10.7845   -0.1260   53.6790   -3.9337    0.2710   -0.3980   65.3661   -0.0354    0.0064   -4.2794
      ⋮
```

```
inv_mat2 = inv(mat2)
```

```
inv_mat2 =
1.0e+03 *

    0.0010   -0.0000         0   -0.0000   -0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000
    0.0459    0.0112   -0.0522   -0.0252    0.0000    0.0012   -0.0022    0.0026   -0.0028    0.0049   -0.0043    0.0029
    0.0000    0.0010    0.0000    0.0000   -0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000
   -0.0408    0.0774   -0.0648    0.0213    0.0010    0.0023   -0.0043    0.0053   -0.0016    0.0033   -0.0088    0.0075
   -0.0000    0.0000    0.0010   -0.0000    0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000
   -0.0014   -0.0076    0.0729    0.0521    0.0003   -0.0007    0.0018   -0.0026    0.0027   -0.0040    0.0021   -0.0022
    0.2212   -0.2423    0.4914    0.1214   -0.0036   -0.0153    0.0305   -0.0364    0.0180   -0.0297    0.0480   -0.0425
    0.0509    0.0527   -0.2607   -0.1438   -0.0000    0.0067   -0.0129    0.0157   -0.0146    0.0225   -0.0199    0.0155
    0.0000   -0.0000    0.0000    0.0010   -0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000
    3.0445   -2.2342    5.1080    1.0512   -0.0454   -0.1939    0.3739   -0.4069    0.1715   -0.2802    0.5271   -0.4995
      ⋮
```

سپس ماتریس های Q به منظور زیر تعریف می شوند.

Q1 : ماتریس تبدیل برای متغیرهای قابل مشاهده مستقیم.

Q2: ماتریس تبدیل برای متغیرهای قابل مشاهده غیرمستقیم.

```
Q_1 = inv_mat2(:,1:q)
```

```
Q_1 =
1.0e+03 *

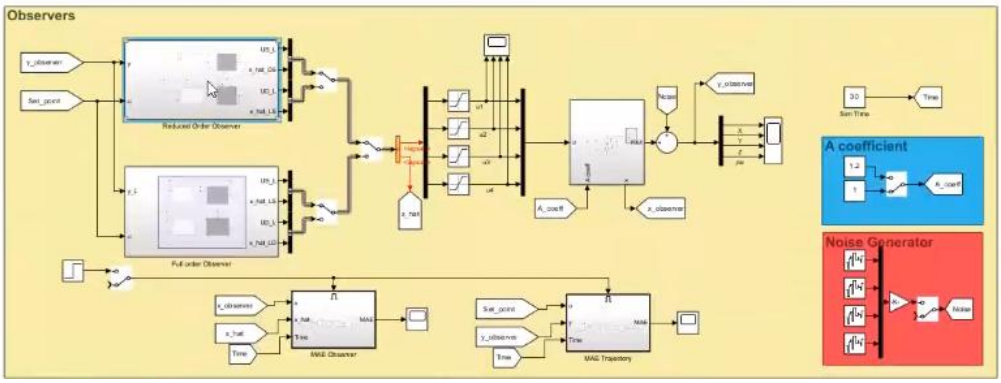
    0.0010   -0.0000         0   -0.0000
    0.0459    0.0112   -0.0522   -0.0252
    0.0000    0.0010    0.0000    0.0000
   -0.0408    0.0774   -0.0648    0.0213
   -0.0000    0.0000    0.0010   -0.0000
   -0.0014   -0.0076    0.0729    0.0521
    0.2212   -0.2423    0.4914    0.1214
    0.0509    0.0527   -0.2607   -0.1438
    0.0000   -0.0000    0.0000    0.0010
    3.0445   -2.2342    5.1080    1.0512
      ⋮
```

```
Q_2 = inv_mat2(:,q+1:n)
```

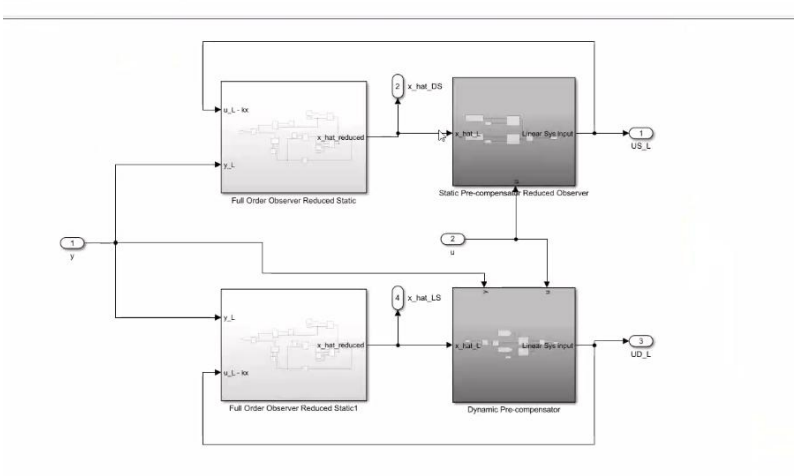
```
Q_2 =
   -0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000
    0.0007    1.2071   -2.2239    2.5972   -2.7933    4.8778   -4.3108    2.9091
   -0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000
    0.9905    2.2976   -4.2930    5.3191   -1.6069    3.3142   -8.7676    7.4726
    0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000
    0.2847   -0.6822    1.8345   -2.5727    2.7487   -3.9997    2.0737   -2.2074
   -3.6004  -15.3239   30.5499  -36.4378   18.0185  -29.7235   47.9512  -42.5394
   -0.0147    6.7080  -12.9448   15.6980  -14.6499   22.4835  -19.9399   15.4784
   -0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000   -0.0000
  -45.3779 -193.9175  373.8574 -406.9437  171.5416 -280.1821  527.1383 -499.5021
     ⋮
```

بلوک observers:

دوتا ضریب بدست آمده از مرحله قبل را در این قسمت جایگذاری کرده و مدل را اجرا می‌کنیم.



redused order observer



فصل سوم:

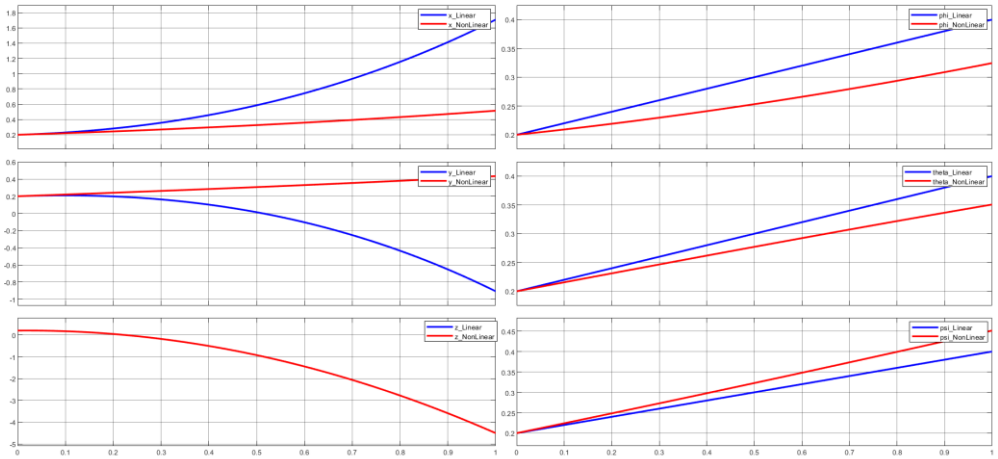
پیاده‌سازی فاز اول و دوم

همان‌طور که در فصل قبل توضیحاتی ارائه شد، به بخش عمده‌ای از سوالات مرتبط با فاز اول و دوم پروژه از طریق کدنویسی در محیط متلب و شبیه‌سازی در سیمولینک پاسخ داده شده است. بنابراین برای جلوگیری از تکرار مطالب، در این فصل تنها به تحلیل و پاسخ به سوالاتی پرداخته می‌شود که نیازمند توضیحات تکمیلی بوده و صرفاً مشاهده نتایج متلب و سیمولینک برای پاسخ‌گویی به آنها کافی نبوده است.

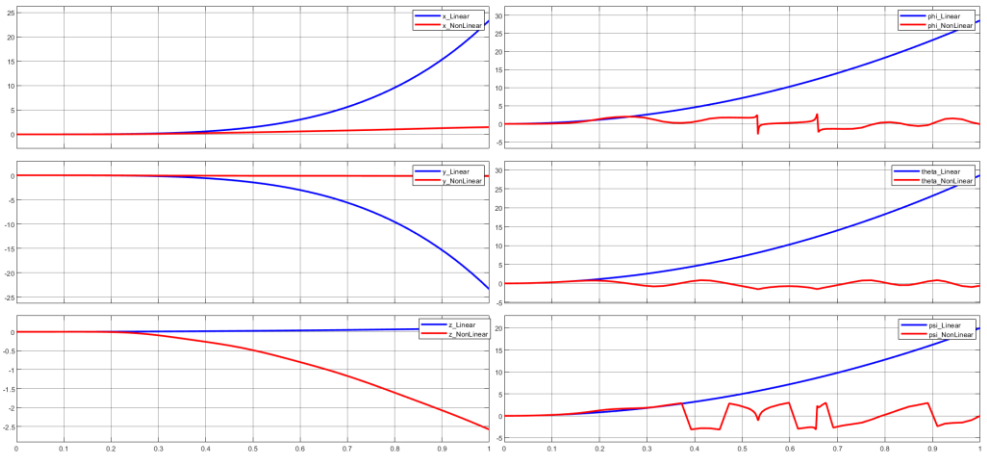
۳-۱- پیاده‌سازی در سیمولینک (پاسخ سوالات ۸ و ۹ فاز اول)

بخش خطی در سیمولینک به دو روش شبیه‌سازی شده‌است. در روش اول، از بلوک State-Space سیمولینک استفاده شده، و در روش دوم از بلاک انتگرال‌گیر برای پیاده‌سازی سیستم بهره بردیم. همچنین، برای سه حالت مختلف که از جنس زاویه بودند، شرطی اعمال کردیم تا مقادیر زاویه فقط در بازه $-\pi$ تا π تغییر کنند. این کار برای نزدیک‌تر شدن به رفتار سیستم غیرخطی انجام شد.

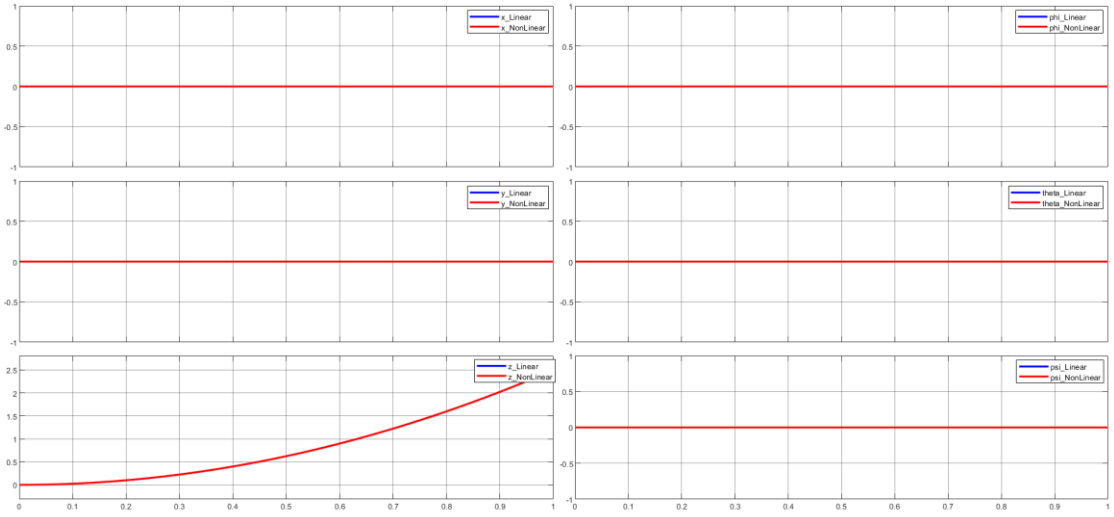
- تشخیص معتبر بودن محدوده خطی: در حالتی که سیستم را خطی کردیم، برای هر مقداری که به u1 بدهیم، در غیرخطی هم دقیق است. درواقع با تغییر u1، می‌توانیم ارتفاع z را با مدل خطی کنترل کنیم. (در شرایط صفر) البته با تغییر شرایط، این حالت دیگر وجود ندارد.



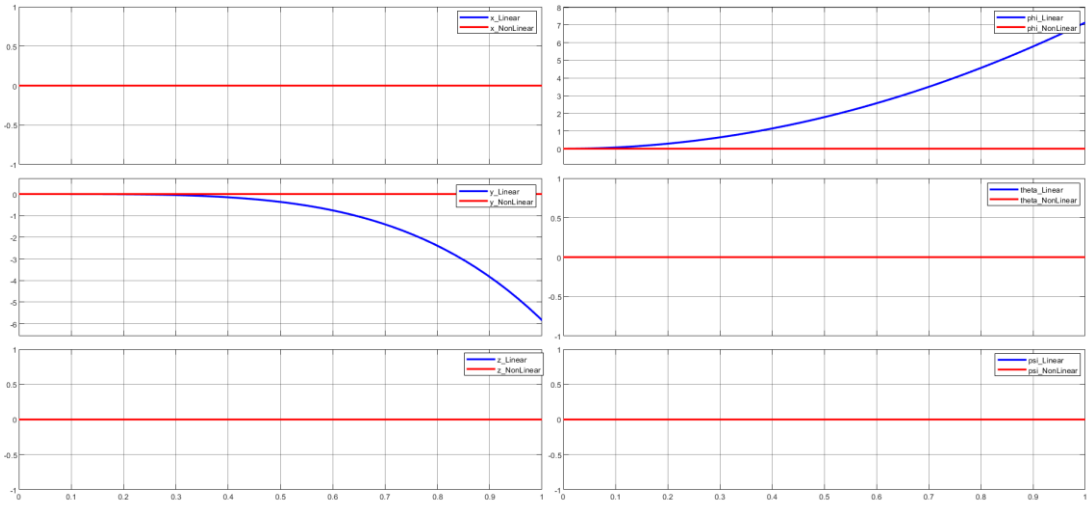
شکل ۱-۳ خروجی سیمولینک برای شرایط اولیه یک و ورودی صفر



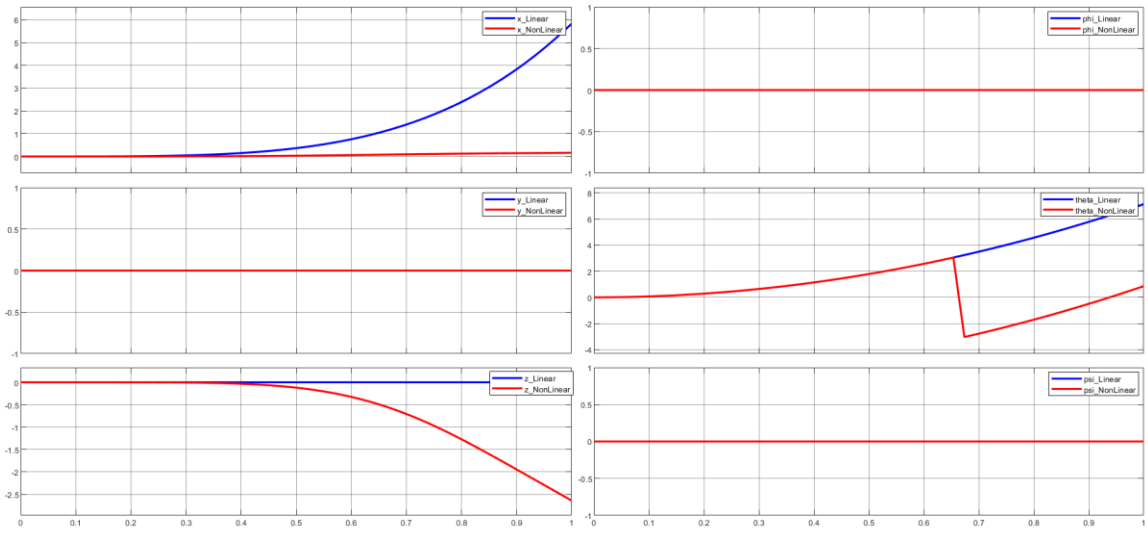
شکل ۲-۳ خروجی سیمولینک برای شرایط اولیه صفر و ورودی ۰.۲



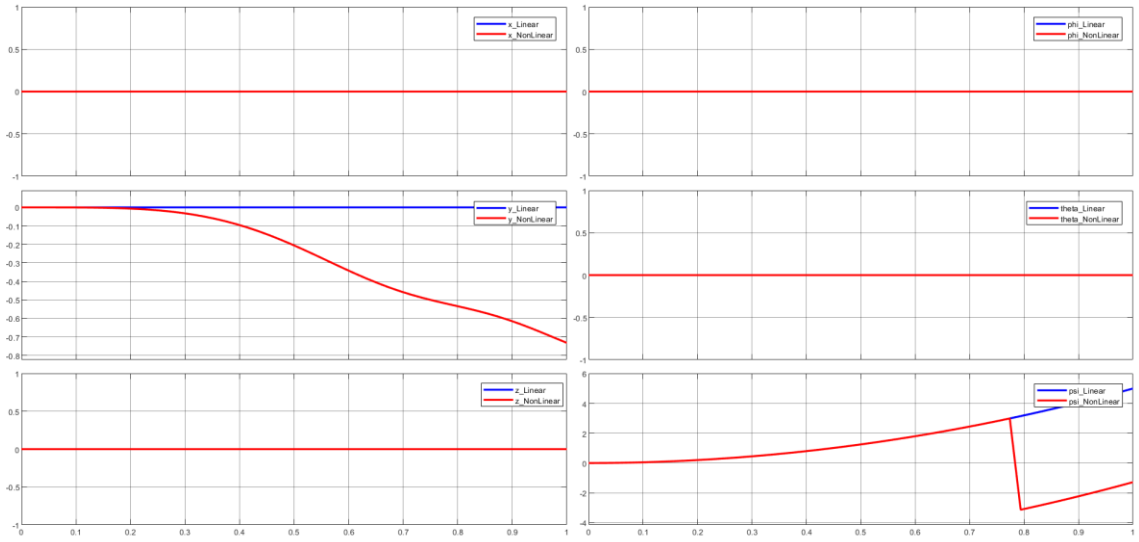
شکل ۳-۳ خروجی سیمولینک برای delta u1 = 5



شکل ۴-۳ خروجی سیمولینک برای delta u2 = 0.05

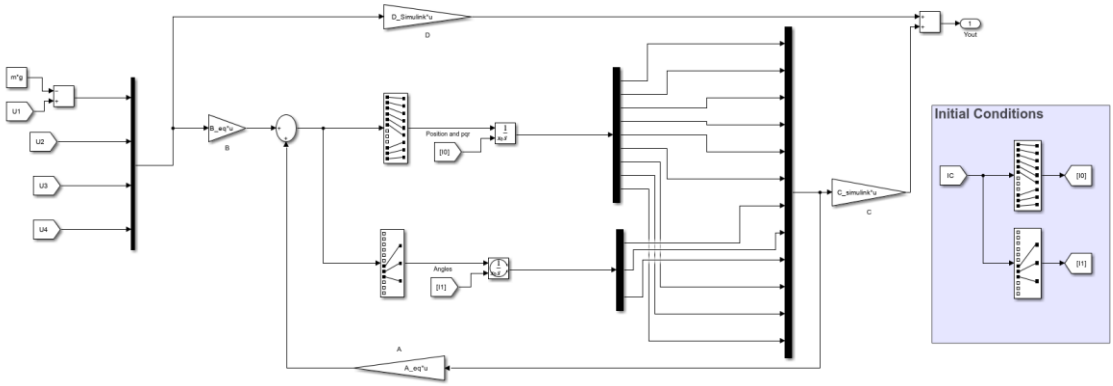


شکل ۵-۳ خروجی سیمولینک برای delta u3 = 0.05

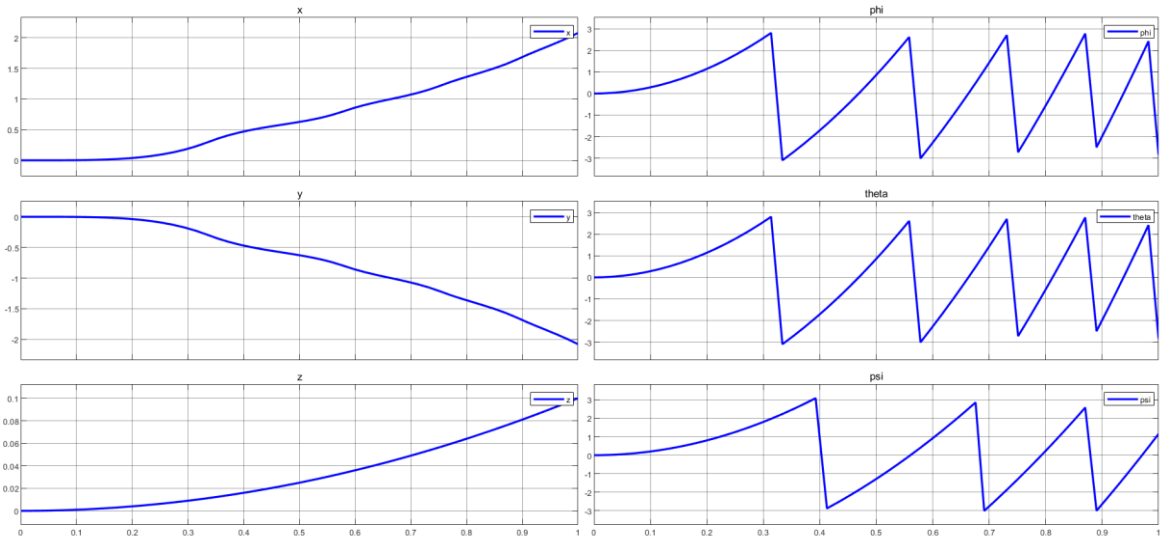


شکل ۶-۳ خروجی سیمولینک برای delta u4 = 0.05

علاوه بر مقایسه‌ها در حالت بلوکی، در سیمولینک به روش دوم نیز به این سوال پاسخ داده‌ایم.



شکل ۷-۳ حالت خطی با انتگرال‌گیر



شکل ۸-۳ حالت خطی با انتگرال‌گیر، شرایط اولیه صفر و ورودی پله ۰.۲

۳-۲- ماتریس انتقال حالت

محاسبه ماتریس انتقال حالت در حوزه لاپلاس: کد ابتدا با استفاده از ماتریس A_eq که پیش‌تر خطی‌سازی شده است، ماتریس انتقال حالت را در حوزه لاپلاس به‌دست می‌آورد. این ماتریس انتقال، که با Phi(s) نشان داده می‌شود Tمحاسبه می‌شود. این ماتریس انتقال حالت در واقع نشان می‌دهد که چگونه حالت‌های سیستم با توجه به ورودی‌ها تغییر می‌کنند و می‌تواند برای پیش‌بینی رفتار سیستم به کار رود.

محاسبه ماتریس انتقال حالت در حوزه زمان: پس از به‌دست آوردن ماتریس انتقال حالت در حوزه لاپلاس، با استفاده از تبدیل لاپلاس معکوس، این ماتریس را به حوزه زمان می‌برد تا ماتریس انتقال حالت Phi(t) به‌دست آید. این ماتریس نشان می‌دهد که چگونه حالات سیستم در طول زمان تغییر می‌کنند و می‌تواند به شکل تابعی از زمان برای پیش‌بینی رفتار دینامیکی سیستم استفاده شود.

تعیین شرایط اولیه و ورودی سیستم در حوزه لاپلاس: شرایط اولیه سیستم به‌صورت یک بردار x0 با مقادیر دلخواه (در اینجا، همه عناصر برابر ۱) تعریف شده است. همچنین، ورودی سیستم به‌صورت یک بردار در حوزه لاپلاس تعریف است. این ورودی‌ها نمایانگر ورودی‌های پله (step inputs) به سیستم هستند.

محاسبه پاسخ حالت‌ها در حوزه لاپلاس و تبدیل آن به زمان: پاسخ سیستم در حوزه لاپلاس با استفاده از ماتریس انتقال حالت، شرایط اولیه، و ورودی‌ها محاسبه می‌شود. عبارت X نشان‌دهنده پاسخ حالت‌ها در حوزه لاپلاس است. سپس با استفاده از تبدیل لاپلاس معکوس، پاسخ سیستم در حوزه زمان محاسبه و به دست می‌آید. این پاسخ به‌طور کلی بیانگر رفتار سیستم با توجه به شرایط اولیه و ورودی‌ها در طول زمان است.

محاسبه خروجی سیستم در حوزه زمان: پس از محاسبه حالت‌های سیستم، خروجی‌های سیستم نیز محاسبه می‌شوند. خروجی‌های سیستم با استفاده از ماتریس C و پاسخ حالت‌ها به‌دست می‌آیند. با تبدیل لاپلاس معکوس، خروجی نهایی در حوزه زمان محاسبه شده و بیانگر پاسخ کامل سیستم به ورودی‌های اعمال‌شده و شرایط اولیه در طول زمان خواهد بود.

خروجی کد:

$$\begin{pmatrix}
\frac{1}{s} & \frac{1}{s^2} & 0 & 0 & 0 & 0 & 0 & \sigma_1 & 0 & 0 & \sigma_2 & 0 \\
0 & \frac{1}{s} & 0 & 0 & 0 & 0 & 0 & \sigma_3 & 0 & 0 & \sigma_1 & 0 \\
0 & 0 & \frac{1}{s} & \frac{1}{s^2} & 0 & 0 & -\sigma_1 & 0 & 0 & -\sigma_2 & 0 & 0 \\
0 & 0 & 0 & \frac{1}{s} & 0 & 0 & -\sigma_3 & 0 & 0 & -\sigma_1 & 0 & 0 \\
0 & 0 & 0 & 0 & \frac{1}{s} & \frac{1}{s^2} & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \frac{1}{s} & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{s} & 0 & 0 & \frac{1}{s^2} & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{s} & 0 & 0 & \frac{1}{s^2} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{s} & 0 & 0 & \frac{1}{s^2} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{s} & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{s} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{s}
\end{pmatrix}$$

where

$$\sigma_1 = \frac{981}{100\,s^3}$$

$$\sigma_2 = \frac{981}{100\,s^4}$$

$$\sigma_3 = \frac{981}{100\,s^2}$$

شکل ۹-۳ ماتریس انتقال حالت در حوزه لاپلاس

$$\begin{pmatrix} 1 & t & 0 & 0 & 0 & 0 & 0 & \sigma_1 & 0 & 0 & \sigma_2 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & \frac{981\,t}{100} & 0 & 0 & \sigma_1 & 0 \\ 0 & 0 & 1 & t & 0 & 0 & -\sigma_1 & 0 & 0 & -\sigma_2 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & -\frac{981\,t}{100} & 0 & 0 & -\sigma_1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & t & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & t & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & t & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & t \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

where

$$\sigma_1 = \frac{981\,t^2}{200}$$

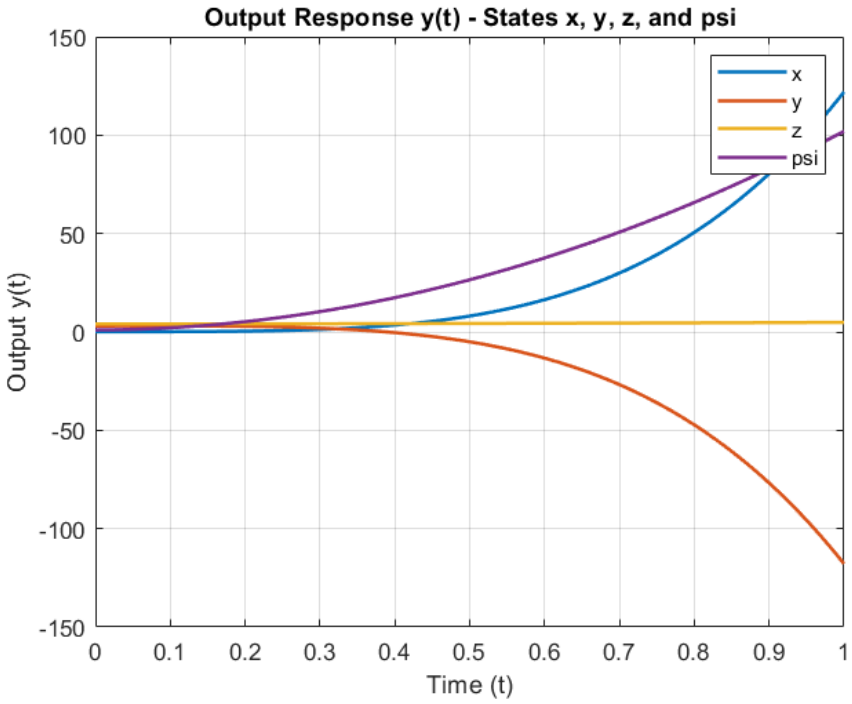
$$\sigma_2 = \frac{327\,t^3}{200}$$

شکل ۱۰-۳ محاسبه ماتریس انتقال حالت در حوزه زمان

y_t =

$$\begin{pmatrix} \frac{1635\,t^4}{14} + \frac{981\,t^3}{200} + \frac{981\,t^2}{20000} + \frac{t}{10} + \frac{1}{10} \\ -\frac{1635\,t^4}{14} - \frac{327\,t^3}{2000} - \frac{981\,t^2}{200} + t + 3 \\ \frac{t^2}{2} + \frac{17\,t}{50} + 4 \\ 100\,t^2 + t + 1 \end{pmatrix}$$

شکل ۱۱-۳ محاسبه خروجی سیستم در حوزه زمان



شکل ۱۲-۳ پاسخ خروجی سیستم به شرط اولیه دلخواه و ورودی پله واحد

۳-۳- تعیین شرایط اولیه (حذف فرکانس)

مشاهده کردیم که سیستم فقط یک مقدارویژه دارد، بنابراین حذف یک فرکانس خاص متناسب با حذف تمام فرکانس‌ها می‌باشد. این حالت، به ازای حالت اولیه صفر اتفاق می‌افتد.

۳-۴- بررسی پایداری BIBO:

سیستمهایی که تمامی مقادیر ویژه آن ها صفر هستند، از نظر BIBO ناپایدار محسوب می شوند . دلیل این امر وجود قطبهایی روی محور موهومی است که باعث رشد نامحدود پاسخ سیستم به برخی ورودی های محدود می شود. به طور خاص، ورودی هایی مانند پله یا سینوسی می توانند خروجی سیستم را به صورت خطی افزایش دهند. این رفتار نشان میدهد که سیستم نمی تواند خروجی خود را در محدوده کنترل شده حفظ کند.

۳-۵- تحلیل حساسیت سیستم نسبت به اغتشاشات

1.حساسیت سیستم در حالت استاتیکی :حساسیت کم و خطای سیستم تقریباً صفر است.

2. .حساسیت سیستم در حالت دینامیکی :سیستم نسبت به تغییرات سریع اغتشاش حساسیت بیشتری نشان می دهد.

➢ در حالت استاتیکی، سیستم رفتار پایدار و بدون خطا دارد. به راحتی به تعادل می رسد و عملکرد سیستم مطلوب است.

➢ در حالت دینامیکی، سیستم به دلیل تغییرات سریع دچار نوسانات، تاخیر و خطا می شود.

فصل چهارم:

فاز سوم

Dynamic Pre-compensator with LQR-۱-۴

در این پروژه، به منظور بهبود عملکرد سیستم کنترل و کاهش هزینه کنترلی، طراحی یک کنترل‌کننده بازخورد حالت بر اساس رویکرد **LQR (Linear Quadratic Regulator)** با قابلیت ردیابی ورودی انجام شده است. برای دستیابی به این هدف، از یک جبران‌ساز دینامیکی جهت بهبود پایداری و دقت پاسخ سیستم استفاده شده است. این روش با حفظ کیفیت پاسخ دینامیکی، هزینه‌های کنترل را بهینه کرده و عملکرد سیستم را در مواجهه با ورودی‌های مختلف ارتقا می‌دهد.

```
Q = diag([10 1 10 1 40 3 10 10 1 1 1 1 500 3500 3000 400 ]);
R = diag([1, 5, 1, 1]);


Ai = [A_eq, zeros(12, 4); -C_final([1 2 3 6], 1:12), zeros(4, 4)];
Bi = [B_eq; zeros(4, 4)];
```

در این طراحی، ماتریس‌های وزنی Q و R به منظور بهینه‌سازی عملکرد سیستم کنترل با استفاده از روش سعی و خطا انتخاب شده‌اند. مقادیر ماتریس Q به گونه‌ای تنظیم شده‌اند که عناصر مربوط به معیارهای کلیدی عملکرد، نظیر دقت و سرعت پاسخ، وزن بیشتری داشته باشند. به‌ویژه، مقادیر بزرگ‌تر عناصر 3000 و 3500 در ماتریس Qنشان‌دهنده اهمیت بالای این متغیرها در طراحی کنترل‌کننده است و هدف از این مقادیر، افزایش سرعت پاسخ سیستم در این متغیرهای کلیدی است. این انتخاب به سیستم امکان می‌دهد تا ضمن حفظ پایداری و دقت، به ورودی‌های مرجع سریع‌تر واکنش نشان دهد. ماتریس R نیز با تنظیم مناسب به منظور کاهش هزینه کنترلی و جلوگیری از اشباع کنترل‌کننده تعیین شده است.

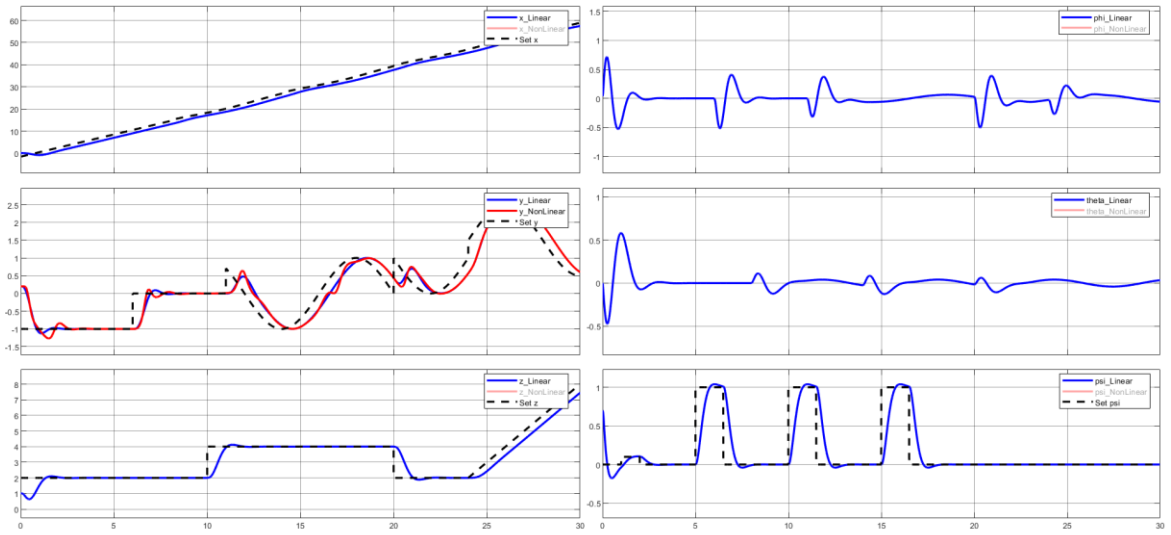
در این طراحی، سیستم جبران‌ساز دینامیکی LQRبه دلیل وجود انتگرال‌گیر داخلی در دینامیک سیستم به‌گونه‌ای اصلاح شده است که یک حالت اضافی به مدل سیستم اضافه شود. این ویژگی باعث تغییر در ساختار ماتریس‌های A، Bو Cشده است. ماتریس‌های جدید Aiو Biبا استفاده از کد محاسبه شده‌اند تا مدل گسترده سیستم، شامل دینامیک انتگرال‌گیر، را در بر گیرد.

```
K_LQR = 4×16
   -0.0000   -0.0000   -0.0000   -0.0000   30.2043    7.9629   -0.0000   -0.0000    0.0000   -0.0000   -0.0000   -0.0000    0.0000    0.0000  -54.7723    0.0000
    0.0000    0.0000  -15.0717   -4.2551    0.0000    0.0000    6.8149    0.0000   -0.0000    0.5627    0.0000    0.0000   -0.0000   26.4575   -0.0000    0.0000
   16.6087    5.9446   -0.0000   -0.0000    0.0000    0.0000    0.0000   11.6150   -0.0000    0.0000    1.0948    0.0000  -22.3607    0.0000   -0.0000   -0.0000
    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000   -0.0000    0.0000    6.6912   -0.0000    0.0000    1.0943   -0.0000   -0.0000   -0.0000  -20.0000
```

P_LQR = 16×1 complex

10^2 ×

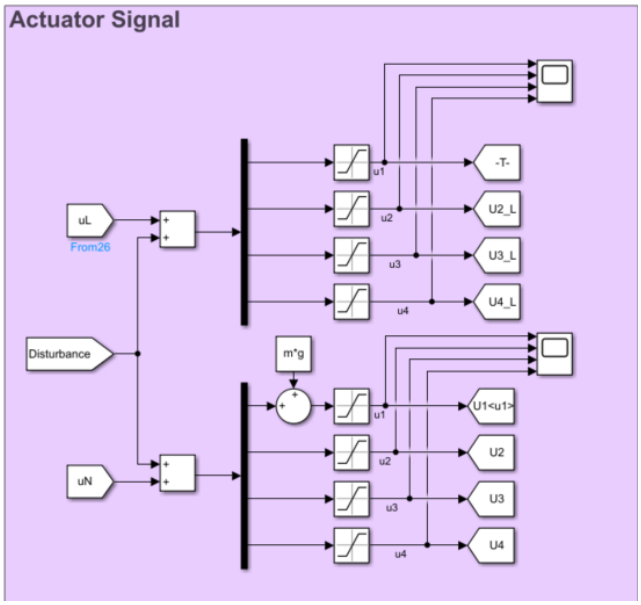
	1
1	-0.0182 + 0.0323i
2	-0.0182 - 0.0323i
3	-0.0206 + 0.0428i
4	-0.0206 - 0.0428i
5	-0.0208 + 0.0317i
6	-0.0208 - 0.0317i
7	-0.0320 + 0.0313i
8	-0.0320 - 0.0313i



شکل ۴ - ۱ خروجی سیستم تحت کنترل بهینه

Actuator Signal -۲-۴

در پاسخ به سوال ۲، برای عملگرهای سیستم یک محدودیت فیزیکی اعمال شده است. این محدودیت برای جلوگیری از تجاوز مقادیر کنترل از محدوده‌های عملیاتی مجاز تعریف شده است. اعمال این محدودیت تضمین می‌کند که سیگنال‌های کنترلی تولیدشده توسط سیستم تحت کنترل LQR در محدوده قابل‌قبول باقی بمانند و بهینه‌سازی عملکرد سیستم در عین حفظ محدودیت‌های فیزیکی ممکن شود.

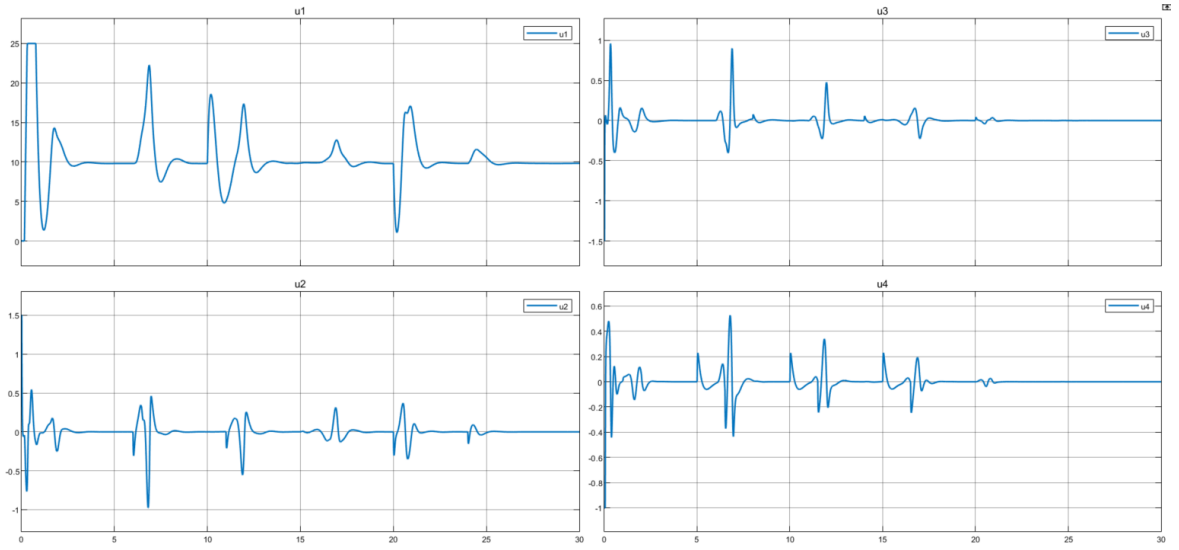


شکل ۴ - ۲بلوک عملگرها

با توجه به اینکه مقدار mg برابر با ۱۰ نیوتن است و موتورها توانایی تولید نیرویی معادل ۲.۵ برابر این مقدار را دارند، محدوده‌های بالا و پایین برای بلوک‌های اشباع با در نظر گرفتن این محدودیت‌های فیزیکی تعیین می‌شوند. این محدوده‌ها بر اساس توزیع نیروی کل تولیدی میان موتورها و تضمین عملکرد بهینه سیستم محاسبه شده‌اند تا مقادیر کنترلی در چارچوب عملیاتی مجاز باقی بمانند.

$$f_t = b\left(\Omega_1^2 + \Omega_2^2 + \Omega_3^2 + \Omega_4^2\right)$$
$$\tau_x = bl\left(\Omega_3^2 - \Omega_1^2\right)$$
$$\tau_y = bl\left(\Omega_4^2 - \Omega_2^2\right)$$
$$\tau_z = d\left(\Omega_2^2 + \Omega_4^2 - \Omega_1^2 - \Omega_3^2\right)$$

در سیستم‌های غیرخطی، به دلیل تعریف نقطه تعادل در حالت تمام صفر، لازم است مقدار mg (نیروی وزن) به ورودی سیستم اضافه شود تا شرایط تعادل حفظ شود. این کار برای جبران اثرات نیروی گرانش و تضمین تعادل سیستم ضروری است.اما در سیستم‌های خطی، به دلیل حضور ترم mg در معادلات خطی‌سازی‌شده و لحاظ آن در مدل، نیازی به افزودن این مقدار به ورودی نیست. در این حالت، نقطه تعادل به طور طبیعی در معادلات خطی‌شده حفظ می‌شود.



شکل ۴ - ۳خروجی سیستم تحت کنترل بهینه

۴-۳- رویتگر ها

۴-۳-۱- رویتگر های مرتبه کامل

رویتگر مرتبه کامل ابزاری در کنترل سیستم‌هاست که تمام حالات داخلی سیستم را تنها از طریق خروجی‌های قابل اندازه‌گیری و ورودی‌ها تخمین می‌زند. این رویتگر با استفاده از مدل فضای حالت سیستم طراحی می‌شود و برای سیستم‌هایی کاربرد دارد که برخی حالات آن‌ها مستقیماً قابل اندازه‌گیری نیستند. هدف اصلی، بازسازی دقیق حالات برای استفاده در طراحی کنترل‌کننده است.

```
C_eq = C_final([1 2 3 6], 1:12);
poles_obsv = [-20 ,-20 , -20 , -21 ,-21,-21,-21.5,-21.5,-21.5,-22,-22,-22];
L = place(A_eq.',C_eq.',poles_obsv).'

A_eq-L*C_eq
```

-0.0084	0.0001	-0.0000	0	0.0000	0	0	0	0.0000	0	0	0
-0.2676	0	-0.0000	0	0.0000	0	0	0.0010	0.0000	0	0	0
0.0000	0	-0.0084	0.0001	0.0000	0	0	0	-0.0000	0	0	0
0.0000	0	-0.2676	0	0.0000	0	-0.0010	0	-0.0000	0	0	0
-0.0000	0	-0.0000	0	-0.0042	0.0001	0	0	0.0001	0	0	0
-0.0000	0	-0.0000	0	-0.0450	0	0	0	0.0017	0	0	0
-0.0000	0	0.3839	0	-0.0000	0	0	0	0.0000	0.0001	0	0
-0.3839	0	-0.0000	0	0.0000	0	0	0	0.0000	0	0.0001	0

در این کد، طراحی رویتگر مرتبه کامل برای سیستم انجام شده است. ابتدا ماتریس مشاهده‌گر با انتخاب سطرهای مناسب از ماتریس خروجی تعریف شده تا ارتباط حالات موردنظر با خروجی سیستم مشخص شود. سپس قطب‌های رویتگر به‌گونه‌ای انتخاب شده‌اند که دینامیک تخمین حالات سریع‌تر از دینامیک سیستم اصلی باشد. در ادامه، بهره رویتگر با استفاده از تابع place محاسبه شده است تا تخمین دقیقی از حالات سیستم فراهم شود. در نهایت، دینامیک جدید رویتگر مشخص می‌شود که بر اساس این بهره و ماتریس مشاهده‌گر، تخمین حالات را تنظیم می‌کند.

۴-۳-۲- رویتگر کاهش یافته

رویتگر مرتبه کاهش‌یافته برای تخمین فقط بخشی از حالات سیستم طراحی می‌شود، به‌ویژه زمانی که برخی حالات مستقیماً قابل اندازه‌گیری باشند. این رویتگر با کاهش ابعاد ماتریس‌های مربوطه، پیچیدگی محاسبات را کاهش می‌دهد و برای سیستم‌هایی با حالات قابل مشاهده و اندازه‌گیری همزمان مناسب است. هدف آن بازسازی دقیق حالات غیرقابل اندازه‌گیری است.

```
F = [ ...
    -20,   1,      1,    -1,   -21,     3,       4,     0;
     0,   -20,    11,    10,     1,     0,       0,     0;
     0,     0,   -20,    20,     1,     1,       1,     0;
     0,     0,     0,   -21,    10,     1,       1,     0;
     0,     0,     0,     0,   -21,    21,       1,     0;
     0,     0,     0,     0,     0,   -21,      10,     0;
     0,     0,     0,     0,     0,     0,   -21.5,    10;
     0,     0,     0,     0,     0,     0,      0,   -21.5
];
L_RO = 100*[3 2 1 5;
     -5 2 2.5 6.3;
     -2 -10 5.6 1.2;
     11 -3 3 5;
     -2 2.5 3 5.6;
     -9 2.5 7 9;
     2 2.75 5 9;
     2 -7 9 2.5]
```

در طراحی رویتگر مرتبه کاهش‌یافته، ابتدا ماتریس‌های Fو Lرا تعریف کردیم که نشان‌دهنده دینامیک سیستم برای بخش غیرقابل اندازه‌گیری و قابل مشاهده هستند. این ماتریس‌ها به‌گونه‌ای تنظیم شدند که ساختار رویتگر بهینه باشد. پس از تعریف این ماتریس‌ها، کنترل‌پذیری آن‌ها را بررسی کردیم تا اطمینان حاصل شود که سیستم می‌تواند به طور کامل توسط رویتگر کنترل و تخمین زده شود.

در مرحله بعد، برای ارزیابی پایداری رویتگر، از تابع لیاپانوف استفاده کردیم. این تحلیل کمک کرد تا مطمئن شویم که دینامیک رویتگر به سمت یک پاسخ پایدار همگرا می‌شود. این مراحل به‌صورت گام‌به‌گام اجرا شد تا عملکرد و دقت رویتگر در تخمین حالات سیستم تضمین شود و اطمینان حاصل شود که طراحی به درستی نیازهای سیستم را برآورده می‌کند.

```
ctrb_mat = ctrb(F,L_RO);
rank_ctr = rank(ctrb_mat)
```

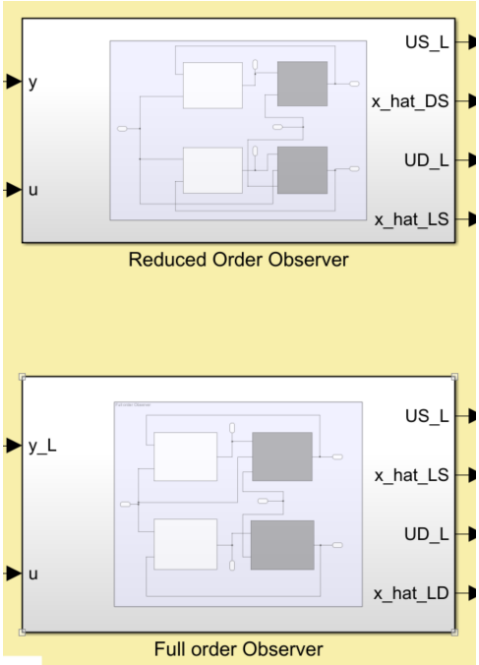
Q_1 = 12×4
10^3 ×

	1	2	3	4
1	0.0010	0	0	0
2	0.0459	0.0112	-0.0522	-0.0252
3	0	0.0010	0	0
4	-0.0408	0.0774	-0.0648	0.0213
5	0	0	0.0010	0
6	-0.0014	-0.0076	0.0729	0.0521
7	0.2212	-0.2423	0.4914	0.1214
8	0.0509	0.0527	-0.2607	-0.1438

Q_2 = 12×8

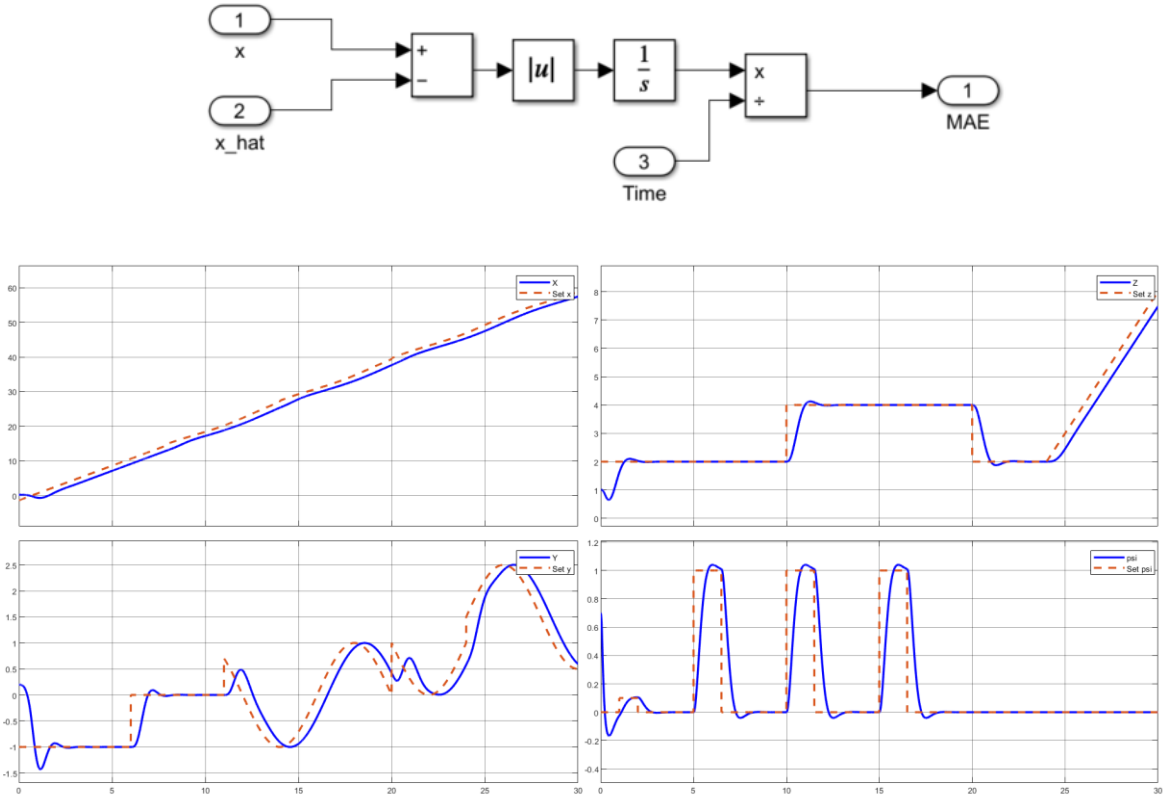
	1	2	3	4	5	6	7	8
1	0	0	0	0	0	0	0	0
2	0.0007	1.2071	-2.2239	2.5972	-2.7933	4.8778	-4.3108	2.9091
3	0	0	0	0	0	0	0	0
4	0.9905	2.2976	-4.2930	5.3191	-1.6069	3.3142	-8.7676	7.4726
5	0	0	0	0	0	0	0	0
6	0.2847	-0.6822	1.8345	-2.5727	2.7487	-3.9997	2.0737	-2.2074
7	-3.6004	-15.3239	30.5499	-36.4378	18.0185	-29.7235	47.9512	-42.5394
8	-0.0147	6.7080	-12.9448	15.6980	-14.6499	22.4835	-19.9399	15.4784

در این مرحله، از ضرایب به‌دست‌آمده برای طراحی رویتگرها در بلوک سیمولینک استفاده کردیم. این ضرایب شامل مقادیر مربوط به رویتگر مرتبه کامل و رویتگر مرتبه کاهش‌یافته است. بلوک سیمولینک برای هر دو نوع رویتگر به‌طور جداگانه پیاده‌سازی شد تا عملکرد آن‌ها در تخمین حالات سیستم بررسی شود. این طراحی امکان مقایسه بین دو روش و ارزیابی دقیق‌تر دینامیک سیستم را فراهم کرد.

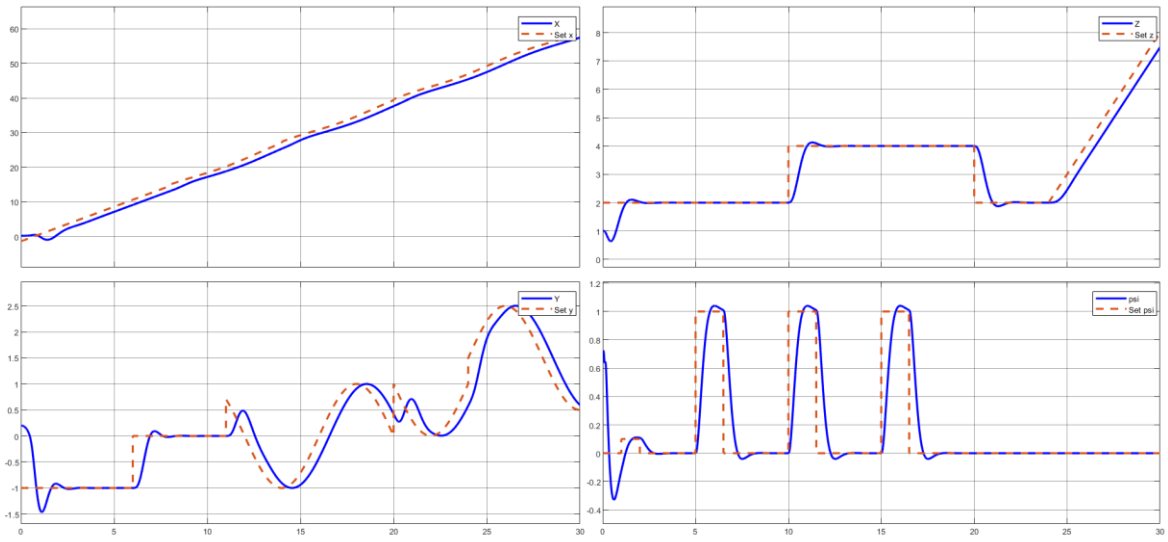


شکل ۴ - ۴ بلوکهای رویت گر کامل و کاهش یافته

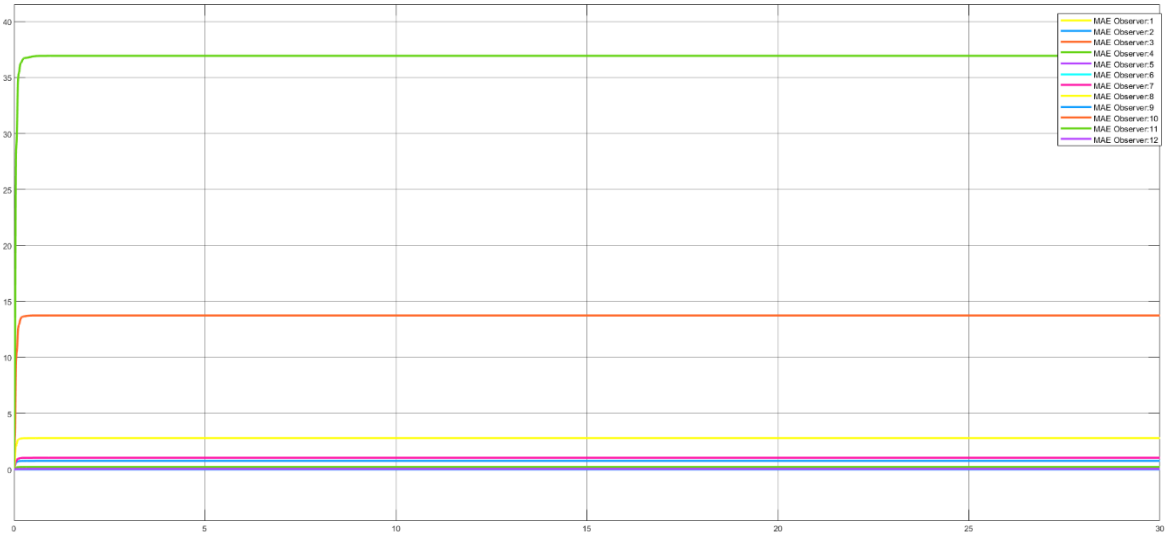
در ادامه، با توجه به تعریف معیار MAE(میانگین قدر مطلق خطا)، بلوک‌های متناظر در سیمولینک ایجاد شدند. این بلوک‌ها برای محاسبه و مقایسه خطای تخمین حالات بین رویتگر مرتبه کامل و رویتگر مرتبه کاهش‌یافته طراحی شدند.



شکل ۴ - ۵خروجی رویت گر مرتبه کامل

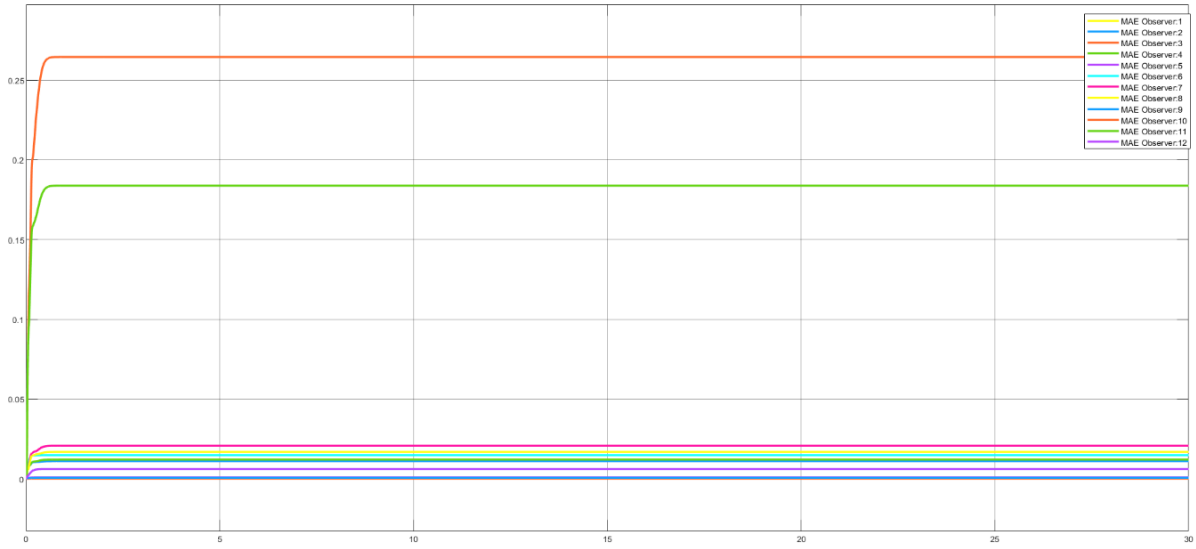


شکل ۴ - ۶خروجی رویت گر کاهش یافته



شکل ۴ - ۷ MAE رویت گر کاهش یافته

این نمودار مربوط به معیار MAE(میانگین قدر مطلق خطا) برای رویتگر مرتبه کاهش‌یافته است و مقادیر خطای تخمین را برای هر یک از حالات سیستم نشان می‌دهد. همان‌طور که مشاهده می‌شود، برخی از حالات سیستم، مانند حالات ۷ و ۹، دارای خطای بالاتری نسبت به سایر حالات هستند. این موضوع می‌تواند به دلیل خصوصیات دینامیکی آن حالات یا حساسیت کمتر رویتگر مرتبه کاهش‌یافته در تخمین این حالات باشد. در مقابل، حالات دیگر دارای مقادیر خطای کمتری هستند، که نشان‌دهنده عملکرد دقیق‌تر رویتگر در این بخش‌هاست.



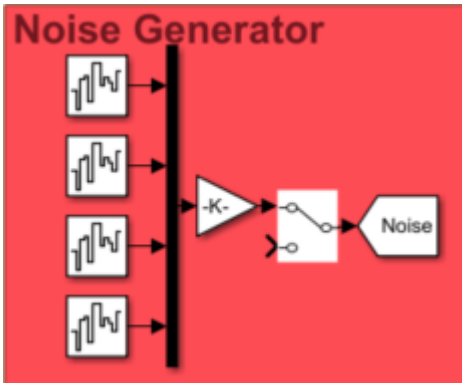
شکل ۴ - MAE۸ رویت گر مرتبه کامل

همان‌طور که مشاهده می‌شود، خطای تخمین در اکثر حالات بسیار کوچک و در حد قابل قبول است که نشان‌دهنده دقت بالای رویتگر مرتبه کامل در بازسازی حالات سیستم است. با این حال، برخی حالات مانند حالت ۷ و ۹ نسبت به سایر حالات خطای بیشتری دارند، که ممکن است به دلیل ویژگی‌های خاص دینامیکی این حالات باشد.

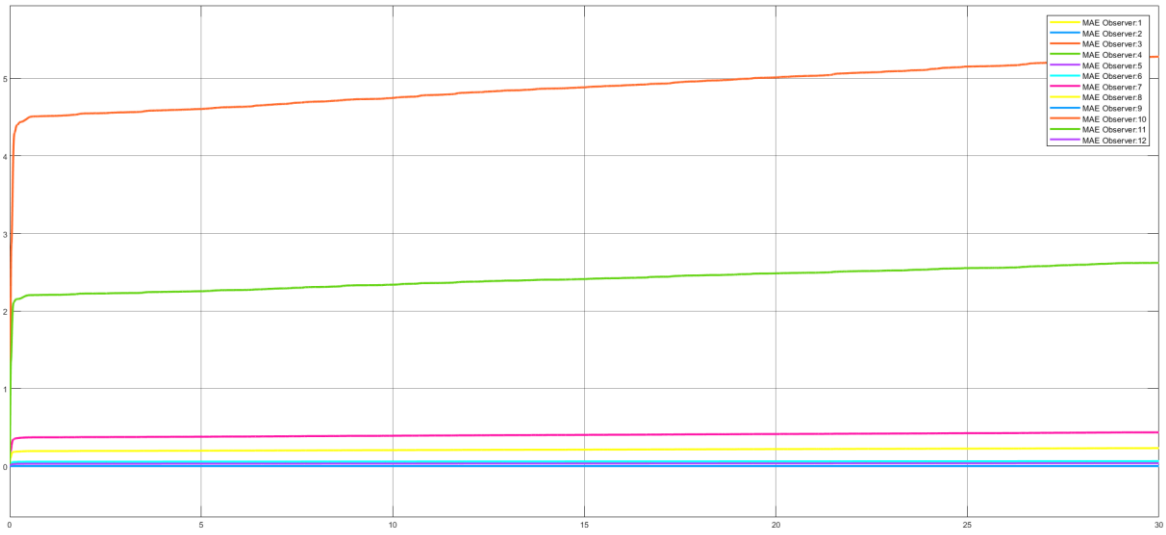
در مقایسه با رویتگر مرتبه کاهش‌یافته، می‌توان نتیجه گرفت که رویتگر مرتبه کامل به‌دلیل دسترسی به تمامی حالات، دقت بالاتری در تخمین داشته و برای کاربردهایی که به تخمین دقیق‌تر نیاز دارند، مناسب‌تر است. این اطلاعات می‌تواند به بهبود طراحی سیستم کنترل و کاهش خطای تخمین کمک کند.

۴-۴- وجود نویز

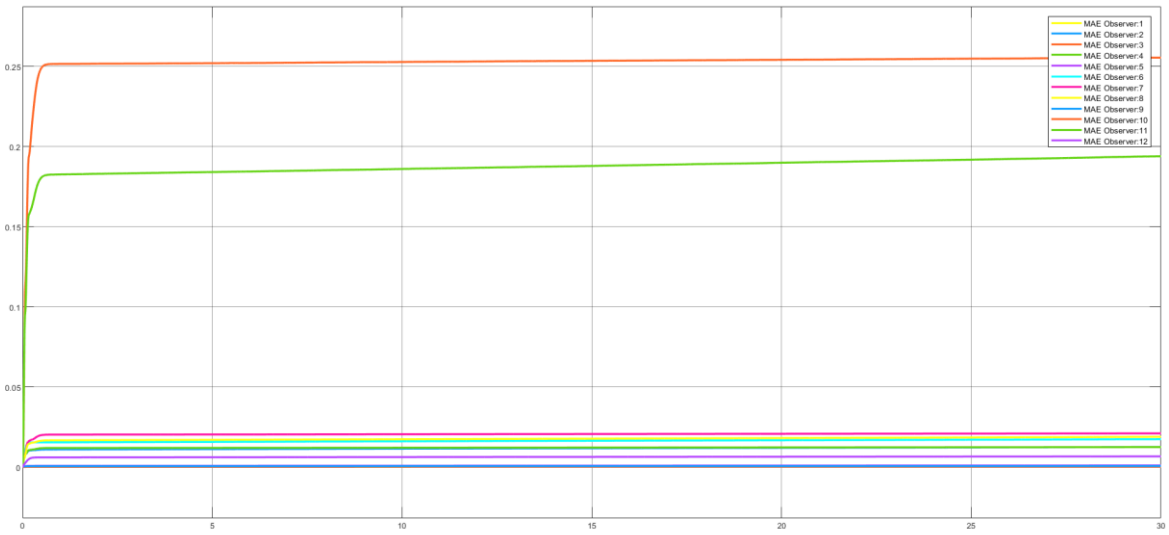
در مرحله بعد، سیستم را با در نظر گرفتن نویز مورد بررسی قرار دادیم تا عملکرد رویتگرها در شرایط غیرایده‌آل ارزیابی شود. برای شبیه‌سازی نویز، از بلوک نویز مشخص‌شده در محیط سیمولینک استفاده کردیم. این بلوک نویز به ورودی‌ها و خروجی‌های سیستم اضافه شد تا شرایط واقعی‌تری شبیه‌سازی شود.



خروجی‌های به‌دست‌آمده پس از اعمال نویز به سیستم به‌صورت زیر است.



شکل ۴ - MAE۹ رویت گر کاهش یافته(نویز)



شکل ۴ – MAE۱۰ رویت گر مرتبه کامل(نویز)

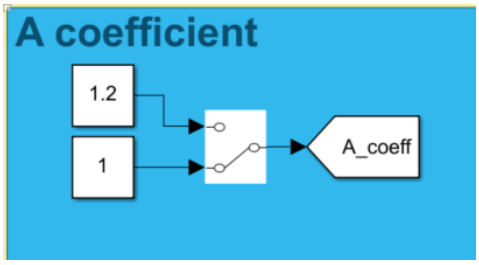
در مقایسه دو نمودار مربوط به رویتگرهای مرتبه کامل و کاهش‌یافته، می‌توان به تفاوت‌های عملکرد این دو روش پی برد. در نمودار مربوط به رویتگر کاهش‌یافته، خطاهای سیستم (MAE) در طول زمان به‌طور قابل‌توجهی افزایش یافته‌اند. این نشان می‌دهد که این رویتگر به دلیل استفاده از مدل ساده‌تر، در مواجهه با نویزهای سیستم دقت کمتری دارد و نمی‌تواند تمامی جزئیات سیستم را به‌درستی تخمین بزند.

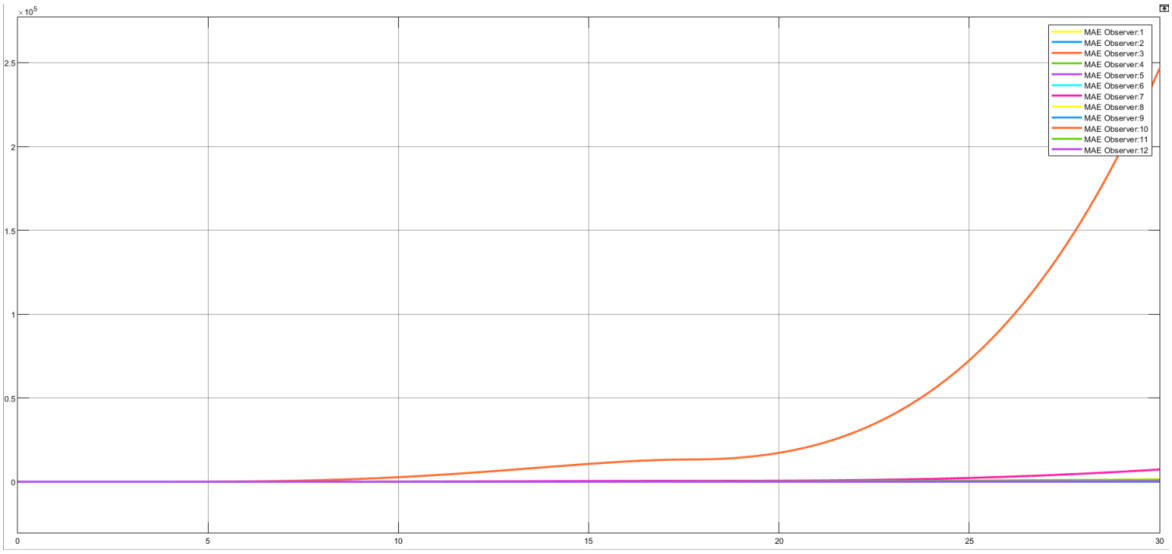
از طرف دیگر، نمودار مربوط به رویتگر مرتبه کامل نشان می‌دهد که خطاها در مقادیر پایینی تثبیت شده‌اند. این موضوع حاکی از آن است که رویتگر مرتبه کامل به دلیل بهره‌گیری از تمامی حالات سیستم و داشتن ساختاری پیچیده‌تر، عملکرد دقیق‌تری داشته و تأثیر نویز را به حداقل رسانده است.

به‌طور کلی، رویتگر کاهش‌یافته برای سیستم‌هایی که منابع محاسباتی محدودی دارند و دقت کامل ضروری نیست، مناسب است. اما در مواردی که دقت بالا اولویت دارد، استفاده از رویتگر مرتبه کامل توصیه می‌شود.

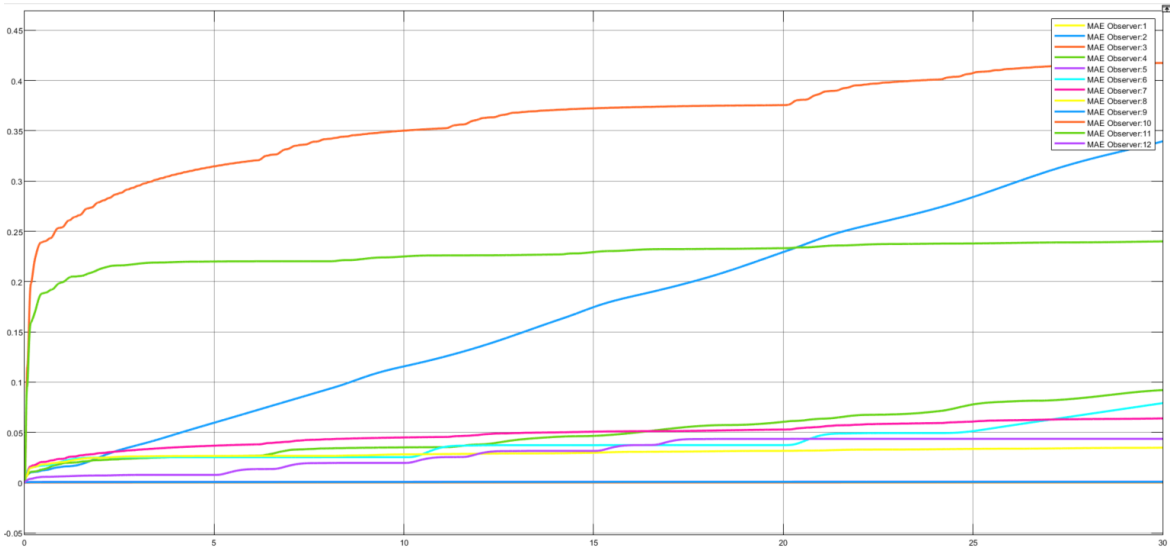
۴-۵- حضور عدم قطعیت های پارامتری

برای بررسی عملکرد کنترل‌کننده و رویتگر در حضور عدم قطعیت‌های پارامتری، مقدار Aرا از ۱ به ۱.۲ تغییر می‌دهیم. سپس خروجی‌های سیستم با استفاده از رویتگرهای مرتبه کامل و کاهش‌یافته مجدداً محاسبه شده و خطای تخمین (MAE) به‌منظور ارزیابی دقت و پایداری سیستم گزارش می‌شود.





شکل ۴ - MAE۱۱ رویت گر مرتبه کاهش یافته(A۱.۲)



شکل ۴ – ۱۲ MAE رویت گر مرتبه کامل)A1.2(

نمودار اول (مرتبه کاهش‌یافته):

در این نمودار، مشاهده می‌شود که برخی ناظران (مانند ناظرهای ۴ و ۵) دارای خطای میانگین مطلق (MAE)رو به رشد و ناپایدار هستند. این رفتار نشان‌دهنده حساسیت زیاد این ناظران به تغییرات پارامتری در سیستم است. ناپایداری و رشد سریع خطاها بیانگر عدم مقاومت (robustness) کافی در طراحی ناظران مرتبه کاهش‌یافته در برابر عدم قطعیت‌های پارامتری است.

نمودار دوم (مرتبه کامل):

در این نمودار، خطای میانگین مطلق (MAE) به‌صورت یکنواخت و با شیب بسیار کم افزایش می‌یابد. این رفتار نشان می‌دهد که ناظران مرتبه کامل در برابر تغییرات پارامتری مقاوم‌تر هستند و پایداری بهتری دارند. با این حال، افزایش خطا با گذشت زمان نشان‌دهنده آن است که هرچند طراحی مرتبه کامل پایدار است، اما همچنان نیاز به بهبود در جهت کاهش تأثیر تغییرات پارامتری وجود دارد.

ناظران مرتبه کامل عملکرد بهتری در مواجهه با تغییرات پارامتری دارند و پایداری بیشتری ارائه می‌دهند. اما برای افزایش دقت و مقاومت بیشتر در برابر تغییرات سیستم، نیاز به بهینه‌سازی و بررسی مجدد طراحی آن‌ها احساس می‌شود.

فصل پنجم:

جمع‌بندی، نتیجه‌گیری و پیشنهادات

جمع‌بندی، نتیجه‌گیری و پیشنهادات

در این پروژه، طراحی و تحلیل سیستم کنترلی کوادروتور با استفاده از روش‌های کنترل انجام شد. با وجود دقت بالای مدل‌سازی و شبیه‌سازی‌ها، پیاده‌سازی عملی این روش‌ها به دلیل برخی محدودیت‌ها و چالش‌های واقعی، مانند عدم توانایی در دستیابی به مدل دینامیکی کاملاً دقیق، بسیار دشوار است. کنترل‌کننده‌های مبتنی بر بازخورد حالت مانند LQR، هرچند در شبیه‌سازی نتایج خوبی ارائه می‌دهند، اما در عمل به دلیل نیاز به داده‌های دقیق و سنسورهای گران‌قیمت، کاربرد عملی محدودی دارند. همچنین، مشاهده‌گر لیونبرگ، به دلیل حساسیت بالا به نویز، در بسیاری از کاربردها عملکرد ضعیفی از خود نشان می‌دهد.

در مجموع، این پروژه نشان داد که روش‌های کنترل می‌توانند به طور چشم‌گیری بهبود عملکرد، پایداری و کارایی سیستم‌های پیچیده را تضمین کنند. با توجه به نتایج به دست آمده، پیاده‌سازی و توسعه روش‌های کنترل مقاوم و بهینه در پروژه‌های صنعتی و پژوهشی آینده پیشنهاد می‌شود.

منابع و مراجع

[1] P. Saraf, M. Gupta, and A. M. Parimi, "A Comparative Study Between a Classical and Optimal Controller for a Quadrotor," Department of Electrical & Electronics Engineering, Birla Institute of Technology and Science – Pilani, Hyderabad, India, 2020.

[2] E. Okyere, A. Bousbaine, G. T. Poyi, A. K. Joseph, and J. M. Andrade, "LQR Controller Design for Quad-Rotor Helicopters," *The Journal of Engineering*, vol. 2019.

[3] F. Ahmad, P. Kumar, and P. P. Patil, "Comparative Analysis of PID and Fuzzy-PID Controllers for Quadcopter Altitude Control," *International Journal of Control Theory and Applications*, vol. 12, no. 4, pp. 55-62, July 2019.